

Deckora

Aplicación web para la optimización de la
experiencia en entornos TCG

Integrantes del grupo

- Anaís Carolina Palma Sánchez
- Vicente Joaquín Verdaguer Gaete

Índice

Índice	2
1. Introducción	5
2. Contexto del proyecto	6
3. Descripción del problema u oportunidad	9
4. Descripción del mercado objetivo	12
4.1 Objetivo general y objetivos específicos	15
5. Situación inicial del cliente (AS-IS)	16
6. Planificación	20
7. Servicios Cloud a utilizar (IaaS / PaaS / SaaS)	22
7.1 Frontend (Hosting)	22
7.2 Backend (API)	22
7.3 Base de Datos	23
7.4 Autenticación	23
7.5 Servicios externos	24
4. Diagrama de infraestructura	24
8. Conceptualización (solución propuesta – versión preliminar)	25
9. Alcance, supuestos y restricciones	30
9.1 Alcance	30
9.2 Entregables	31
9.3 Supuestos	31
9.4 Restricciones	31
10. Metodología de desarrollo y gestión del proyecto	32
10.1 Metodología utilizada	32
10.2 Roles y responsabilidades	33
10.3 Definition of Done (DoD)	34
11. Documentos y diagramas de diseño de la solución	36
11.1 Casos de uso / Historias de usuario	36
11.2 Diagrama de flujo de usuario (User Flow)	38
11.3 Wireframes / Mockups de pantallas principales	41
11.4 Modelo de datos (MER / DER)	47
11.5 Arquitectura de alto nivel	49
12. Arquitectura de software y patrones de diseño	51
12.1 Arquitectura utilizada	51
12.2 Patrones de diseño aplicados	52
12.3 Diagrama de componentes	54
12.4 Justificación de la arquitectura	55
13.1 Lenguaje principal	56
13.2 Frontend	56
13.3 Backend	57
13.4 Base de datos e infraestructura	58
13.5 Estándares de código	59

13.6 Repositorios y flujo de trabajo	59
14. Configuración de servidor de producción (paso a paso)	64
14.1 Plataformas elegidas	64
14.2 Requisitos técnicos del servidor	64
14.3 Variables de entorno backend (configuradas en Render):	65
14.4 Variables de entorno frontend (configuradas en Vercel):	65
14.5 Instrucciones paso a paso	65
14.6 Logs y monitoreo básico	67
15. Estado de avance del desarrollo	70
15.1 Calidad y seguridad aplicadas	73
16. Integraciones necesarias	74
16.1 Supabase Auth — Autenticación de usuarios	74
16.2 Nomic AI — Embeddings vectoriales	75
16.3 DeepSeek (Deepseek-V4-flash) — Explicaciones en lenguaje natural	75
16.4 Mapbox — Mapas interactivos	76
16.5 Resend — Correos transaccionales	77
17. Estrategia y enfoque de calidad	78
17.2 Atributos de calidad evaluados (ISO/IEC 25010)	79
17.3 Niveles y tipos de prueba (orden de aplicación)	80
17.4 Enfoque	81
17.5 Definición de "Hecho" (Definition of Done)	81
18. Plan de pruebas	82
18.1 Alcance de las pruebas	82
18.2 Elementos y funcionalidades a probar	82
18.3 Criterios de aceptación / rechazo	83
18.4 Criterios de suspensión y reanudación	83
18.5 Recursos, roles y cronograma (matriz RACI)	84
18.6 Premisas, dependencias y riesgos	85
19. Tipos de pruebas aplicadas	86
19.1 Pruebas unitarias y de componentes	86
19.2 Pruebas de humo (smoke)	86
19.3 Pruebas de integración	87
19.4 Pruebas funcionales / API (contrato HTTP)	87
19.5 Pruebas de extremo a extremo (E2E)	87
19.6 Pruebas de regresión	88
19.7 Pruebas de rendimiento / carga	88
19.8 Pruebas de seguridad	88
20. Herramientas y software utilizados	89
20.1 Herramienta de gestión documental de pruebas	90
21. Ejecución de pruebas y resultados	91
21.1 Evidencia de pruebas y resultados	92
21.1.1 Ejecución de la suite del backend	93
21.1.2 Panel de cobertura del backend	93
21.1.3 Ejecución de la suite del frontend	94

21.1.4 Panel de cobertura frontend	94
21.1.5 Reporte de playwright (E2E)	95
21.1.6 Métricas de carga (k6)	96
21.1.7 Auditoría de dependencias	96
21.1.8 Análisis estadístico de seguridad	97
22. Conclusión	98
22.1 Beneficios esperados	99
22.2 Próximos pasos	100
22.3 Objetivos a corto plazo: Preparación para producción	100
22.4 Objetivos a mediano plazo: Escalabilidad y robustez	101
22.3 Objetivos a largo plazo: Adopción y evolución del producto	101
23. Anexos	102

1. Introducción

El crecimiento de los juegos de cartas coleccionables (TCG) ha dado lugar a comunidades activas de jugadores y organizadores que participan tanto en torneos casuales como competitivos. Sin embargo, gran parte de la información relevante para desenvolverse en este ámbito se encuentra distribuida en múltiples plataformas, como redes sociales o grupos privados, lo que dificulta el acceso, especialmente para nuevos jugadores. A esto se suma la falta de herramientas integradas que permitan gestionar de manera eficiente colecciones, mazos y participación en torneos, generando una experiencia fragmentada dentro del ecosistema TCG.

En este contexto, se identifica como problemática principal la dispersión de la información y la ausencia de una solución centralizada que permita a los usuarios gestionar sus recursos y participar activamente en la comunidad. Actualmente, la organización de torneos, el seguimiento del rendimiento de los jugadores y la construcción de mazos se realizan de forma manual o mediante herramientas poco conectadas entre sí. Además, la baja visibilidad de tiendas y eventos limita la participación y el crecimiento de la comunidad. Frente a esta situación, el proyecto propone el desarrollo de **Deckora**, una plataforma web orientada a centralizar la gestión de colecciones, creación de mazos y organización de torneos, incorporando además herramientas de análisis y recomendaciones que apoyen la toma de decisiones de los usuarios.

El presente informe aborda de manera estructurada los distintos aspectos del desarrollo del proyecto. En primer lugar, se presenta el contexto del proyecto junto con la descripción del problema u oportunidad identificada y el análisis del mercado objetivo. Posteriormente, se definen el objetivo general y los objetivos específicos que guían la solución propuesta. A continuación, se describe la situación inicial (AS-IS) y la planificación del proyecto. Luego, se detallan los servicios Cloud a utilizar y la conceptualización de la solución en su versión preliminar. Finalmente, se establecen el alcance, los supuestos y las restricciones del proyecto, junto con su conclusión y los anexos correspondientes, permitiendo una comprensión integral del trabajo desarrollado.

2. Contexto del proyecto

El crecimiento de los juegos de cartas coleccionables (TCG) ha generado una comunidad cada vez más activa, tanto a nivel recreativo como competitivo. Sin embargo, este crecimiento no ha ido acompañado de soluciones integrales que faciliten la participación dentro del ecosistema. Actualmente, los jugadores deben recurrir a múltiples plataformas para realizar distintas acciones, como gestionar sus mazos, informarse sobre torneos o contactar con otros jugadores. Esta fragmentación dificulta la experiencia, especialmente para quienes buscan integrarse o mejorar su desempeño, evidenciando la necesidad de herramientas más completas y accesibles que respondan a las dinámicas actuales del entorno TCG.

El origen de esta problemática surge a partir de la observación directa y la experiencia dentro de la comunidad TCG. Se identifica que gran parte de la información relevante, como la organización de torneos o la ubicación de tiendas, se difunde principalmente a través de redes sociales como Facebook o grupos cerrados de mensajería, lo que limita su alcance. Además, la gestión de mazos y el seguimiento del rendimiento de los jugadores suele realizarse mediante herramientas separadas o incluso de forma manual, lo que dificulta la toma de decisiones informadas. Esta situación no solo afecta a jugadores nuevos, sino también a usuarios con mayor experiencia que buscan optimizar su rendimiento o participar de manera más activa en el entorno competitivo.

El público objetivo de este proyecto corresponde, de manera general, a jugadores de TCG que desean gestionar sus colecciones, construir y mejorar sus mazos, así como participar en torneos. Asimismo, se considera a organizadores y tiendas especializadas que requieren herramientas para gestionar torneos, registrar resultados y dar visibilidad a sus actividades. Ambos perfiles comparten la

necesidad de contar con una plataforma centralizada que simplifique procesos y mejore la interacción dentro de la comunidad.

En la actualidad, existen diversas soluciones que abordan parcialmente estas necesidades. Por ejemplo, plataformas como Moxfield permiten la creación y análisis de mazos, mientras que TopDeck.gg se enfoca en la organización de torneos. Por otro lado, aplicaciones como MTG Companion ofrecen funcionalidades oficiales para eventos específicos. Sin embargo, estas herramientas presentan limitaciones importantes, ya que se enfocan en aspectos aislados del ecosistema y no entregan una solución integrada. Además, algunas funcionalidades como la búsqueda de tiendas o eventos se encuentran limitadas geográficamente o no están disponibles en todos los contextos, lo que reduce su utilidad para comunidades locales fuera de mercados principales como Estados Unidos por dar un ejemplo.

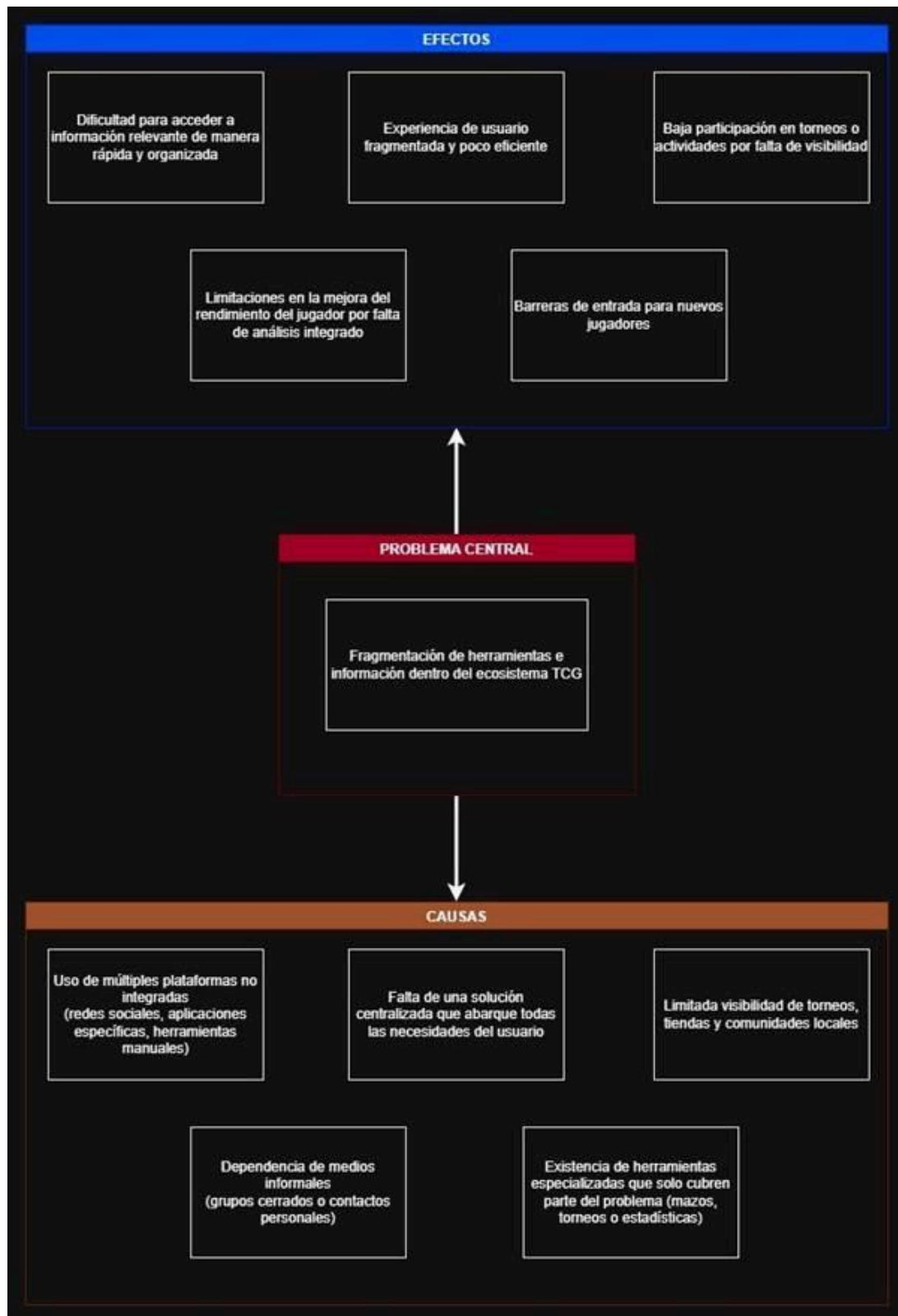
En este contexto, la oportunidad radica en desarrollar una solución que unifique estas funcionalidades en una sola plataforma, permitiendo no solo la gestión de mazos y torneos, sino también la visualización de estadísticas, recomendaciones basadas en datos y la integración de un mapa interactivo de tiendas y eventos con mayor cobertura. Esta propuesta busca superar la fragmentación actual, facilitando tanto la participación como el desarrollo competitivo de los usuarios dentro del entorno TCG.

Solución / Competidor	Qué hace	Ventajas	Desventajas	Qué hará distinto Deckora
Moxfield	Creación y gestión de mazos	Interfaz intuitiva, análisis de cartas	No gestiona torneos ni comunidad	Integrar mazos + torneos + estadísticas
TopDeck.gg	Organización de torneos	Facilita gestión competitiva	No incluye deckbuilding ni recomendaciones	Unificar torneos con análisis de rendimiento
MTG Companion	Gestión de eventos oficiales	Integración con torneos de Magic	Limitado a un solo juego y ecosistema	Ser multi-TCG y más flexible
Redes sociales (Facebook, WhatsApp)	Difusión de torneos y contacto	Alta masividad	Información desordenada y poco accesible	Centralizar información en una sola plataforma

3. Descripción del problema u oportunidad

Actualmente, la información y las herramientas necesarias para participar activamente en el entorno de los juegos de cartas coleccionables (TCG) se encuentran fragmentadas en múltiples plataformas, lo que provoca dificultades en la gestión de mazos, la organización de torneos y el acceso a la comunidad, generando una experiencia poco integrada para los usuarios.

Esta situación impacta directamente en la experiencia de los jugadores y organizadores, ya que implica una mayor inversión de tiempo para encontrar información relevante, como torneos o lugares donde jugar, además de generar dependencia de redes sociales o contactos personales. Asimismo, la gestión manual o mediante herramientas separadas aumenta la probabilidad de errores en el registro de resultados o en la construcción de mazos. En términos de costos, si bien no existe un impacto económico directo significativo, sí se traduce en una menor eficiencia, pérdida de oportunidades de participación y una experiencia general menos accesible, especialmente para nuevos usuarios que desean integrarse a la comunidad.



1. Diagrama de árbol

Este análisis permite identificar que la principal oportunidad radica en la centralización de funcionalidades y la integración de herramientas dentro de una única plataforma. En este contexto, la solución propuesta mediante Deckora se orienta a abordar directamente la fragmentación existente, permitiendo unificar la gestión de mazos, la organización de torneos, el análisis de desempeño y la visualización de información relevante en un solo sistema, mejorando así la accesibilidad, la participación y la experiencia general de los usuarios dentro del ecosistema TCG.

4. Descripción del mercado objetivo

El mercado objetivo al cual se orienta la solución propuesta corresponde principalmente a los usuarios que participan en el entorno de los juegos de cartas coleccionables (TCG), quienes requieren herramientas que les permitan gestionar sus colecciones, optimizar y construir sus mazos, así como informarse y participar en eventos y torneos. En segundo lugar, se considera también como parte del mercado objetivo a las tiendas especializadas y a los organizadores independientes de torneos, quienes necesitan herramientas que faciliten la planificación, gestión y difusión de eventos dentro de la comunidad.

A continuación se presentan perfiles representativos de los usuarios objetivos de nuestra plataforma:

Usuario 1: Jugador de TCG

Usuario principal. Corresponde a una persona que participa activamente en juegos de cartas coleccionables, cuyo nivel puede variar entre casual y competitivo. Generalmente utiliza múltiples plataformas y fuentes dispersas para informarse, además de herramientas externas para la gestión de mazos. Una de sus principales necesidades es contar con una plataforma que centralice la información, herramientas, mejore su rendimiento y permita el fácil acceso a eventos dentro de su comunidad.

Necesidades:

- Plataforma que centralice la información
- Acceso a estadísticas
- Herramientas de gestión de mazos

Dolores:

- Información dispersa en múltiples plataformas (redes sociales, apps, grupos)
- Dificultad para encontrar torneos y eventos relevantes
- Falta de herramientas integradas para analizar rendimiento

- Pérdida de tiempo al tener que usar varias aplicaciones
- Dificultad para mantenerse actualizado dentro de la comunidad

Escenario de uso:

- Acceso a la página web de la aplicación
- Consulta de torneos y eventos disponibles antes de participar
- Gestión de mazos y revisión de estadísticas previo a partidas

Usuario 2: Organizador de torneos

Usuario secundario. Corresponde a personas enfocadas en la planificación y ejecución de torneos, con un enfoque más estructurado y, en la mayoría de los casos, competitivo. Estos organizadores buscan herramientas que permitan una gestión expedita de inscripciones, manejo de estadísticas, además del registro y control de resultados. Su principal necesidad es optimizar la organización de torneos y mantener la fluidez de estos.

Necesidades:

- Gestión de inscripciones
- Registro y control de resultados
- Generación de enfrentamientos
- Visualización de estadísticas
- Difusión de torneos

Dolores:

- Uso de herramientas manuales o poco especializadas
- Dificultad para organizar torneos de forma eficiente
- Baja visibilidad de los eventos organizados

Escenario de uso:

- Uso durante la planificación y ejecución de torneos
- Registro de jugadores e inscripciones previo al evento
- Gestión de enfrentamientos y resultados en tiempo real

- Publicación y difusión de torneos

Usuario 3: Tiendas

Usuario secundario. Incluye a establecimientos que, además de comercializar productos relacionados con TCG, funcionan como puntos de encuentro para comunidades asociadas. Este usuario, al igual que el anterior, organiza torneos, pero además cuenta con eventos fuera de un ambiente competitivo, como instancias de iniciación, demostraciones y actividades casuales. Su principal necesidad es gestionar distintos tipos de eventos, mejorar la comunicación con los jugadores y aumentar la visibilidad de sus actividades.

Necesidades:

- Gestión de distintos tipos de eventos
- Difusión de actividades
- Aumento de visibilidad
- Herramientas de organización

Dolores:

- Dificultad para atraer nuevos jugadores
- Baja visibilidad de eventos y actividades
- Dependencia de redes sociales para difusión
- Desorganización en la gestión de eventos
- Falta de herramientas adaptadas a distintos tipos de actividades

Escenario de uso:

- Uso en la planificación de eventos y actividades
- Publicación y difusión de torneos y eventos comunitarios
- Gestión de distintos tipos de eventos (competitivos y casuales)

4.1 Objetivo general y objetivos específicos

Objetivo general

Desarrollar una plataforma web que permita centralizar la gestión de colecciones, mazos y torneos en juegos de cartas coleccionables (TCG), incorporando herramientas de análisis y recomendaciones, con el fin de mejorar la experiencia, participación y toma de decisiones de jugadores y organizadores dentro de la comunidad.

Objetivos específicos

- **Objetivo 1 — Diseño de base de datos** Diseñar y documentar la estructura completa de la base de datos del sistema durante el Sprint 1 (semanas 1 y 2), contemplando un mínimo de 5 entidades principales —usuarios, cartas, mazos, torneos y resultados— con el 100% de sus relaciones, restricciones e índices definidos, formalizada en un modelo entidad-relación que sea revisado y aprobado formalmente por el equipo mediante una reunión de validación realizada antes del inicio del Sprint 2, sin observaciones pendientes registradas al momento del cierre.
- **Objetivo 2 — Módulos de colecciones y mazos** Implementar, durante el Sprint 2 (semanas 3 a 4), los módulos de backend y frontend que permitan a los usuarios registrar, editar y organizar sus colecciones de cartas y construir mazos personalizados, de modo que al menos el 90% de las funcionalidades definidas en el backlog del sprint queden operativas y validadas mediante un protocolo de pruebas internas documentado —con casos de prueba explícitos y resultados registrados— antes del cierre formal del Sprint 3.
- **Objetivo 3 — Funcionalidades de torneos** Desarrollar e integrar, dentro del Sprint 3 (semanas 5 y 6), las funcionalidades de creación, administración y participación en torneos, incluyendo el registro de resultados y la generación

automática de rankings, de manera que antes del inicio de la fase de pruebas sea posible ejecutar al menos un torneo de prueba completo —de inicio a fin— registrando 0 errores críticos durante el flujo principal y logrando que el 100% de las etapas del torneo —inscripción, registro de resultados y generación de ranking— se completen sin interrupciones.

- **Objetivo 4 — Integración de IA y estadísticas** Integrar antes del cierre del Sprint 3 (5 y 6) y validar durante el Sprint 4 (semana 7 y 8) al menos una herramienta basada en inteligencia artificial que permita visualizar estadísticas de desempeño del jugador —tasa de victoria, uso de cartas y rendimiento por mazo— y generar recomendaciones automáticas de mejora, alcanzando una tasa de aprobación mínima del 80% en las pruebas funcionales y de usabilidad ejecutadas durante la semana de pruebas, con los resultados registrados en un informe que documente los casos evaluados y los criterios de aceptación utilizados.

5. Situación inicial del cliente (AS-IS)

Actualmente, los jugadores y organizadores de juegos de cartas coleccionables (TCG) utilizan múltiples herramientas y plataformas para gestionar sus actividades dentro de la comunidad. Este proceso no se encuentra centralizado, por lo que implica una serie de pasos manuales y el uso de distintas aplicaciones para completar tareas que podrían estar integradas en una sola solución.

En el caso de los jugadores, el proceso comienza cuando el usuario desea participar en un torneo o mejorar su desempeño dentro del juego. Para ello, primero debe buscar información sobre eventos disponibles, lo cual generalmente se realiza a través de redes sociales como Facebook, grupos de WhatsApp o contactos personales. Luego, el usuario debe recopilar información relevante del torneo, como fecha, ubicación y formato, muchas veces de manera desorganizada o incompleta.

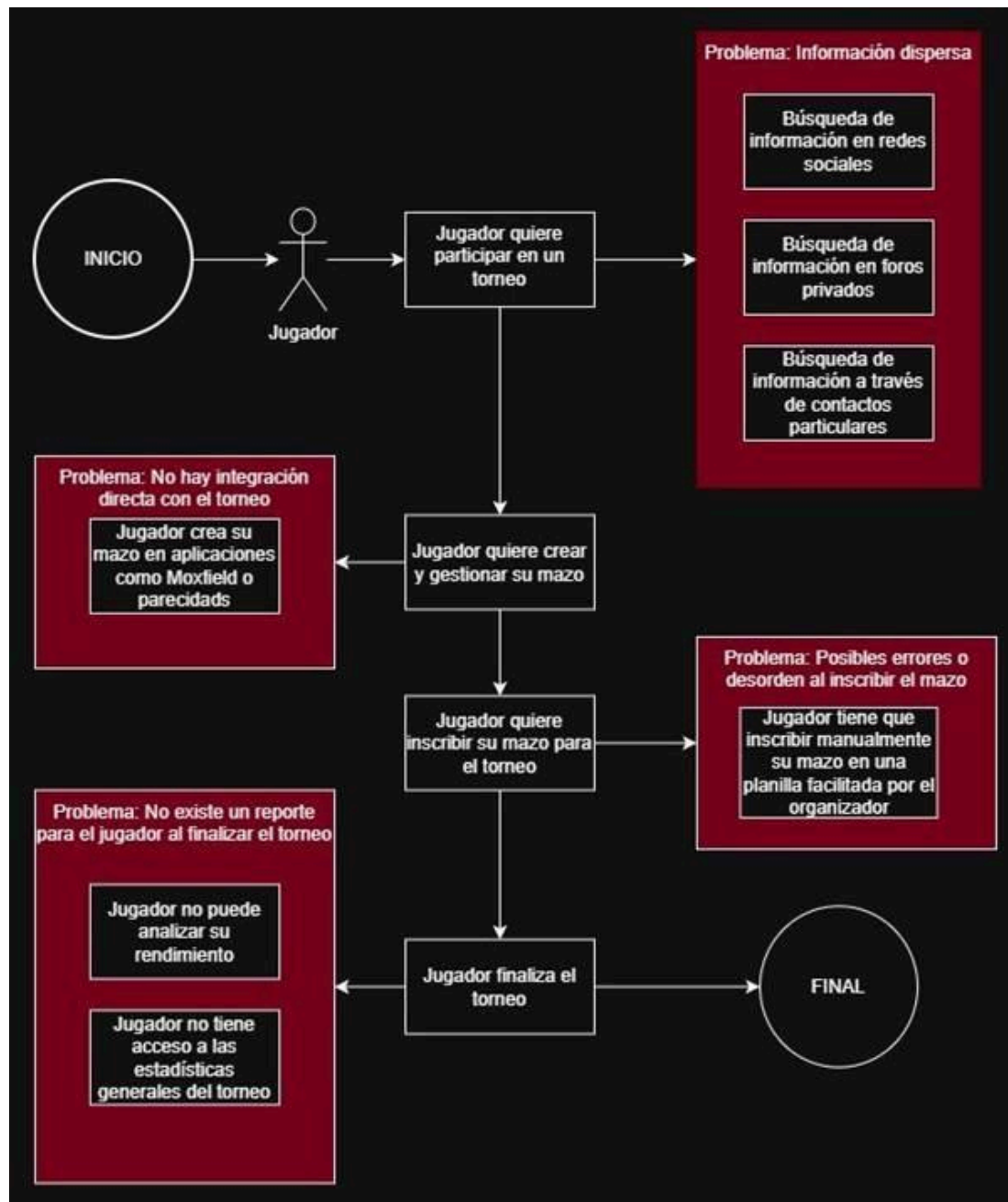
Posteriormente, el jugador gestiona su mazo utilizando herramientas externas como Moxfield u otras aplicaciones similares, las cuales permiten la construcción de mazos, pero no se encuentran integradas con sistemas de torneos o estadísticas.

En paralelo, el usuario no cuenta con una fuente centralizada que le permita analizar su rendimiento, por lo que el seguimiento de resultados suele hacerse de forma manual o simplemente no se realiza.

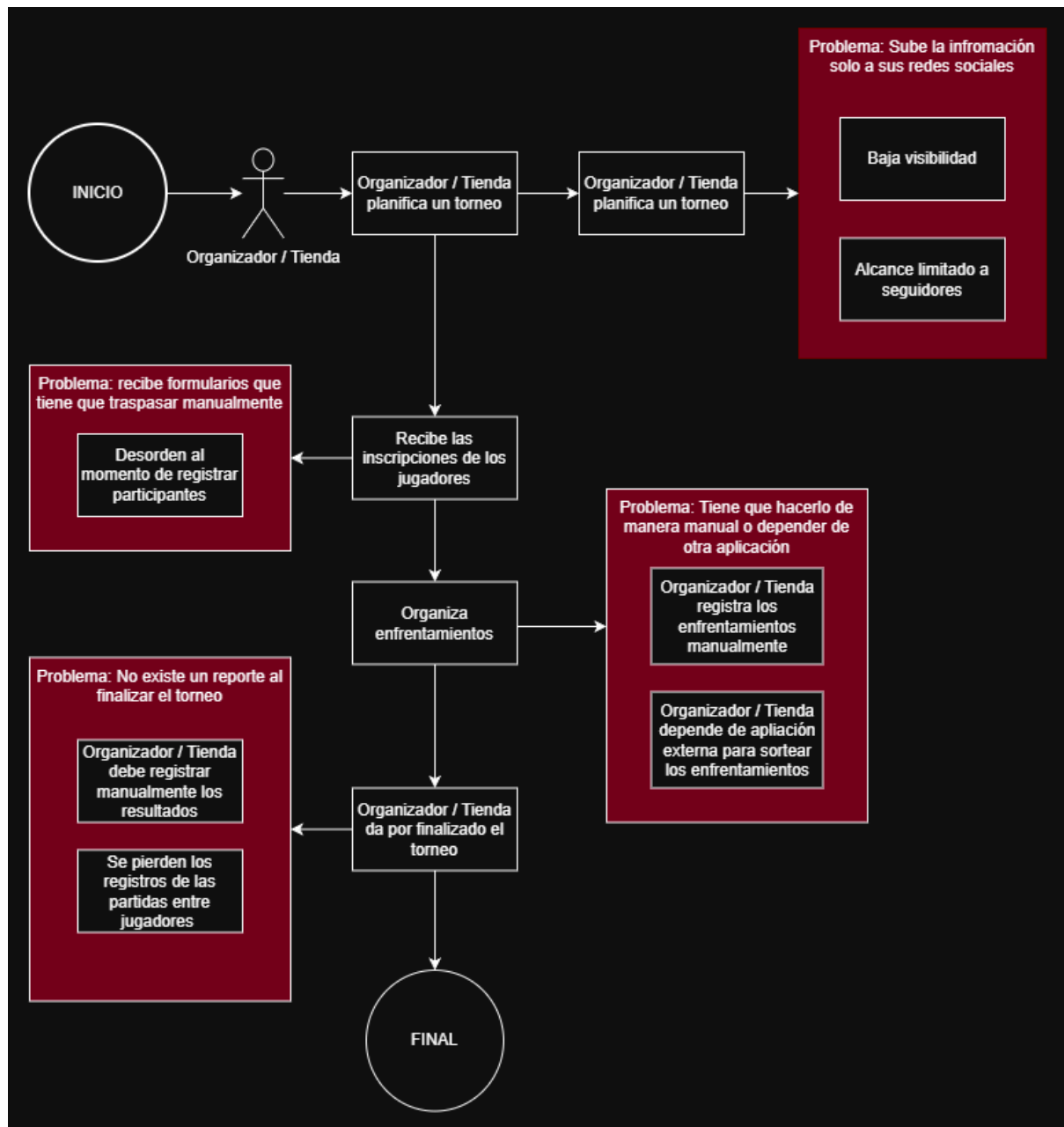
En el caso de los organizadores o tiendas, el proceso comienza con la planificación de un torneo. Para ello, deben difundir el evento utilizando redes sociales, lo que limita el alcance a su círculo cercano o seguidores. Luego, las inscripciones se gestionan manualmente o mediante formularios externos, lo que puede generar desorden o errores en el registro de participantes.

Durante el desarrollo del torneo, los organizadores deben gestionar enfrentamientos y resultados, muchas veces utilizando herramientas poco especializadas o registros manuales, lo que aumenta la probabilidad de errores. Finalmente, la información del torneo no siempre queda registrada de manera estructurada, dificultando su posterior análisis o consulta por parte de los jugadores.

Los principales problemas de este proceso se generan en distintos puntos críticos. En primer lugar, la búsqueda de información se ve afectada por la dispersión de plataformas, lo que dificulta el acceso a eventos y comunidades. En segundo lugar, la gestión de mazos y el seguimiento del rendimiento se realizan en herramientas separadas, impidiendo una visión integral del desempeño del jugador. Además, la organización de torneos presenta fallas debido al uso de métodos manuales o poco integrados, lo que puede derivar en errores y pérdida de información. Finalmente, la baja visibilidad de tiendas y eventos limita la participación y el crecimiento de la comunidad.



2. Diagrama As-Is perfil de jugador



3. Diagrama As-Is perfil de organizador/tienda

6. Planificación

Organizaremos el proyecto de manera que se desarrolle en distintas fases, abarcando desde la definición inicial hasta la etapa final de entrega. Dentro de estas fases se consideran actividades específicas, asignación de tareas, responsabilidades y una estimación del tiempo necesario para cada una, con el fin de asegurar el cumplimiento de los objetivos dentro del periodo establecido.

La planificación se estructura en sprints, organizando el trabajo en 4 sprints de 2 semanas cada uno, lo que permite dividir el desarrollo en periodos de tiempo más manejables.

Las responsabilidades de cada integrante del equipo se distribuyen de acuerdo con los roles asumidos por cada uno, dividiéndose principalmente en desarrollo de backend y frontend, además de trabajo colaborativo en las etapas críticas donde es necesario tomar decisiones.

A continuación, se presentan las principales actividades del proyecto, junto con el encargado, duración estimada y una breve descripción.

Sprint	Actividades	Duración	Descripción general
1	- Definición del proyecto - Análisis del problema - Análisis de requisitos - Definición de alcance y objetivos	Semana 1 y Semana 2	Definición y análisis. Establecimiento de las bases del proyecto mediante la definición, análisis del problema, requisitos y objetivos.
2	- Diseño de la arquitectura - Diseño de la base de datos - Diseño interfaz	Semana 3 y Semana 4	Diseño de la solución. Definición de la estructura del sistema, incluyendo arquitectura, base de datos e interfaz de usuario.
3	- Desarrollo backend - Desarrollo frontend - Integración del sistema	Semana 5, Semana 6 y Semana 7	Desarrollo e integración. Implementación del backend, frontend e integración de los componentes del sistema.
4	- Pruebas - Documentación final - Entrega y presentación final	Semana 6 y Semana 7	Validación y cierre. Realización de pruebas, documentación del proyecto y preparación de la entrega final.

Para complementar la planificación, se incluye una Carta Gantt como documento adjunto que muestra de manera visual las tareas a realizar a lo largo de las 8 semanas consideradas para el proyecto. En ella se presentan tanto las tareas a detalle, como el orden en que se desarrollarán.

7. Servicios Cloud a utilizar (IaaS / PaaS / SaaS)

Para el desarrollo e implementación de la solución propuesta, se utilizarán distintos servicios en la nube que permiten asegurar disponibilidad, escalabilidad y facilidad de despliegue. La arquitectura se basa principalmente en el uso de plataformas PaaS y SaaS lo cual resulta adecuado para el alcance del proyecto.

El uso de servicios cloud permite reducir costos operativos, facilitar el despliegue continuo de la aplicación y asegurar alta disponibilidad, además de permitir escalar los recursos en caso de aumento de usuarios o carga en el sistema.

7.1 Frontend (Hosting)

Para el despliegue del frontend se utilizará **Vercel**, plataforma de tipo PaaS especializada en aplicaciones web modernas.

Se selecciona esta herramienta debido a su integración nativa con aplicaciones desarrolladas en React, permitiendo despliegues automáticos, manejo de dominios, optimización de rendimiento y distribución mediante CDN. Esto facilita la entrega de contenido de forma rápida y eficiente a los usuarios, mejorando la experiencia de uso.

7.2 Backend (API)

El backend será desarrollado en Node.js con Express y desplegado en Render que es un servicio PaaS, lo que permite ejecutar la API sin necesidad de gestionar servidores físicos o máquinas virtuales.

El uso de una plataforma PaaS para el backend permite:

- simplificar el despliegue y mantenimiento
- escalar la aplicación según demanda
- centralizar la lógica de negocio del sistema

Además, esta capa se organiza de manera modular, permitiendo separar responsabilidades sin necesidad de implementar una arquitectura de microservicios completa.

7.3 Base de Datos

Para la gestión de datos se utilizará **Supabase**, que provee una base de datos relacional basada en **PostgreSQL**.

Se selecciona esta solución debido a:

- su facilidad de integración con aplicaciones web
- soporte para consultas complejas mediante SQL
- escalabilidad y disponibilidad en la nube
- herramientas integradas para administración de datos

Adicionalmente, se utilizará la extensión **pgvector**, que permite almacenar y consultar embeddings, facilitando la implementación de funcionalidades de análisis y recomendación dentro del sistema.

7.4 Autenticación

La autenticación será gestionada mediante el servicio integrado de Supabase, lo que corresponde a un modelo SaaS.

Este servicio permite:

- registro e inicio de sesión de usuarios
- gestión de sesiones mediante tokens (JWT)
- integración segura con la base de datos

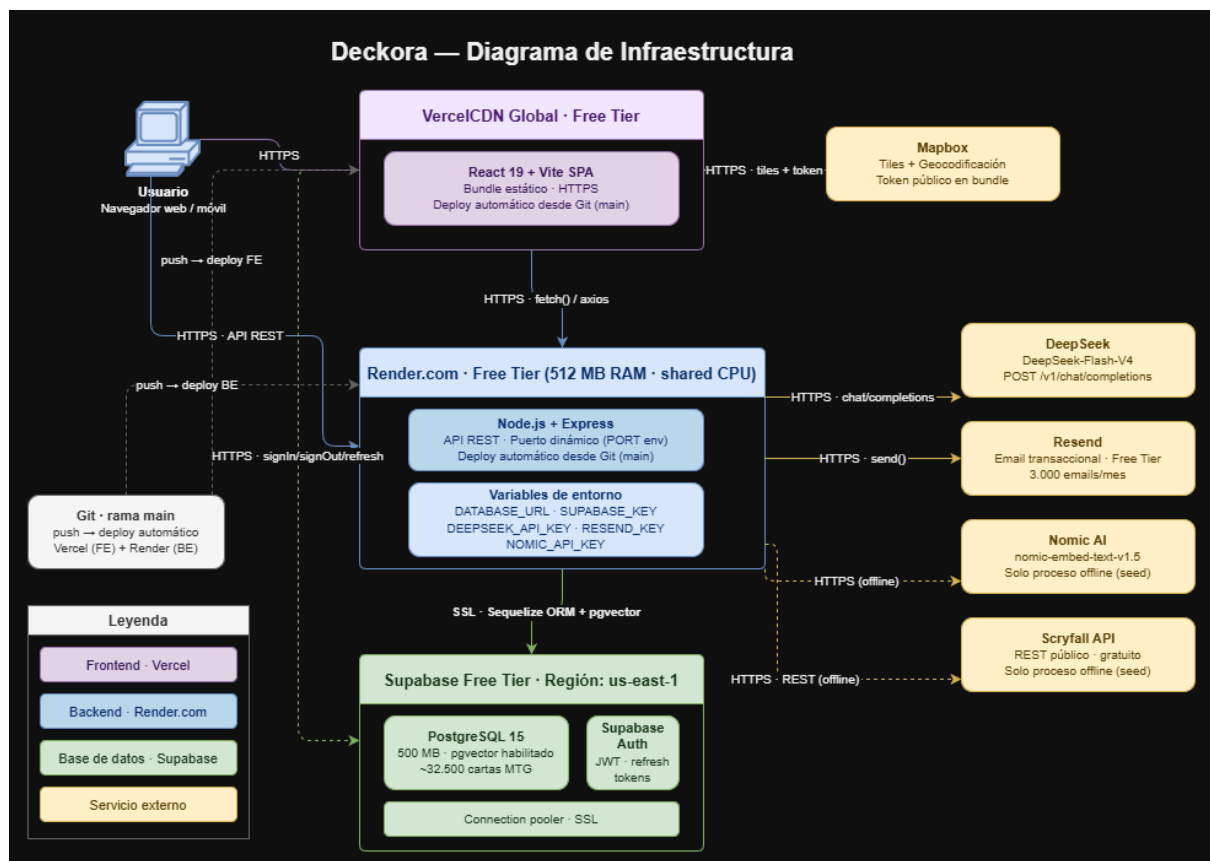
Su uso permite evitar la implementación de un sistema de autenticación propio, reduciendo la complejidad del desarrollo y mejorando la seguridad de la aplicación.

7.5 Servicios externos

El sistema también integrará servicios externos de tipo SaaS para complementar funcionalidades específicas:

- **Scryfall API** para la obtención de información sobre las cartas.
- **Resend** para el envío de notificaciones por correo electrónico.
- **Nomic AI** para el embedding de las cartas
- **Deepseek** para las recomendaciones de cartas con ia, para las explicaciones del por qué esa carta y el autocompletado de mazos.

Estos servicios permiten enriquecer la funcionalidad del sistema sin necesidad de desarrollar dichas capacidades desde cero.



4. Diagrama de infraestructura

8. Conceptualización (solución propuesta – versión preliminar)

La solución propuesta por Deckora consiste en una plataforma web que elimina la fragmentación dentro del entorno TCG. A diferencia de la situación actual, donde la información, comunidad y herramientas están dispersas a través de distintas plataformas, Deckora centraliza estos mismos dentro de una misma plataforma.

Este nuevo proceso comienza por personalizar la experiencia del usuario, al iniciar sesión, el sistema obtiene la localización del usuario, ofreciendo a este una cartelera de eventos cercanos y basados puramente en las preferencias de él mismo. La construcción de mazos deja de ser una tarea aislada, integrándose directamente dentro de la plataforma, entregando así mismo un asistente manejado por IA que realiza sugerencias basadas en desempeño anterior así como las tendencias actuales (meta). Finalmente, la implementación de herramientas que facilitan tanto la inscripción, resultados y manejo de estadísticas dentro de eventos competitivos.

A continuación, nos enfocaremos en en las funcionalidades principales, basados en los objetivos y alcance del proyecto:

- **Sistema de autenticación y personalización:** Módulo de gestión de usuarios que permite el inicio de sesión seguro y la detección de ubicación para filtrar las tiendas, eventos y torneos más cercanos al jugador.
- **Construcción de mazos:** Herramienta de creación, edición y gestión de mazos integrada con APIs externas para obtener información técnica de las cartas en tiempo real.
- **Módulo de recomendaciones con IA:** Motor de sugerencias que identifica las cartas presentes de un mazo y genera sugerencias en base a las tendencias competitivas actuales y el desempeño anterior dentro de eventos, en caso de haber registros.
- **Historial de desempeño:** Panel que muestre un historial del rendimiento del jugador dentro de distintos eventos, permitiendo un seguimiento de la evolución competitiva del mismo

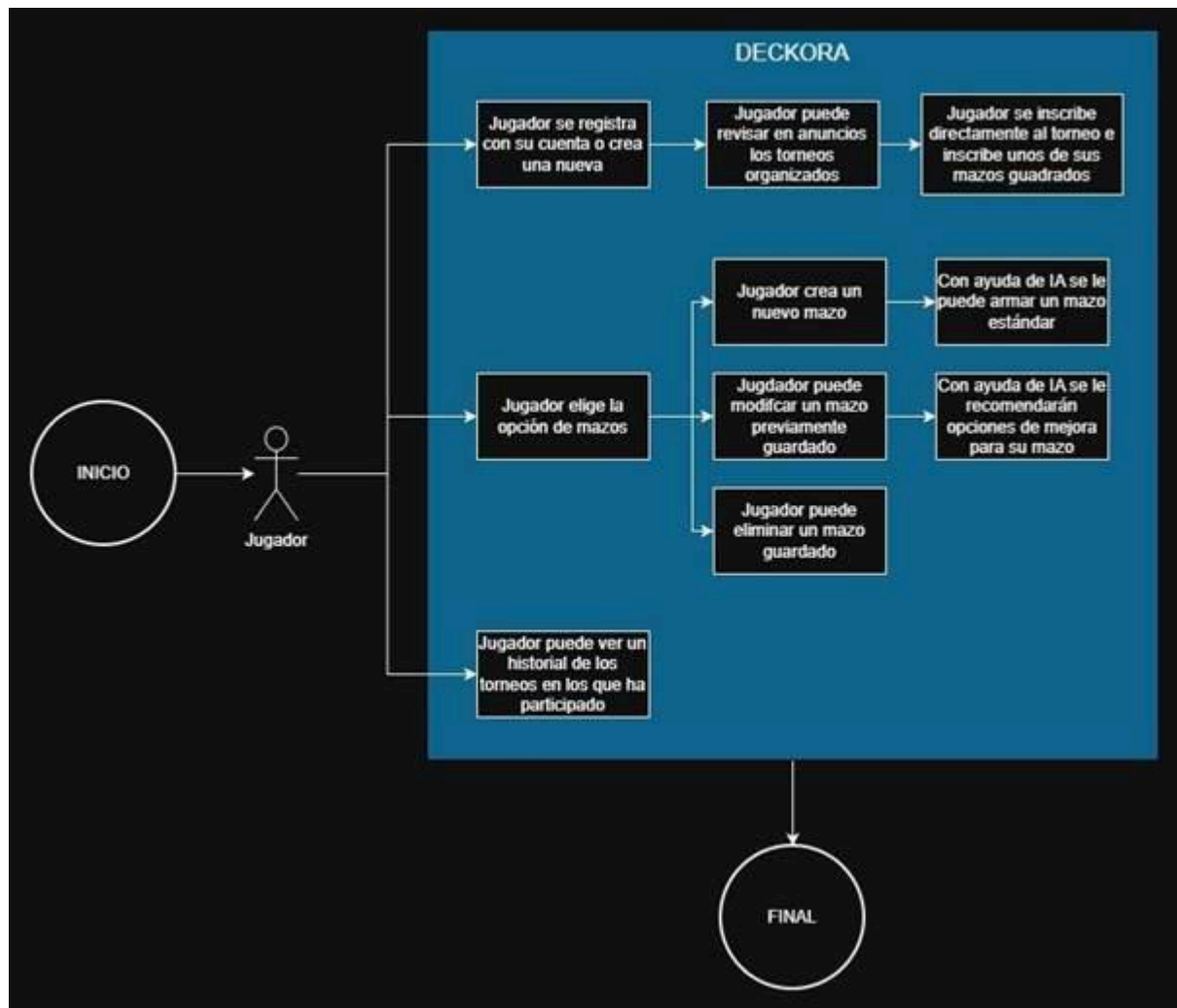
- **Gestor de torneos y eventos:** Panel para que organizadores publiquen eventos y jugadores puedan inscribirse, con generación automática de enfrentamientos y tablas de posiciones.

Como resultado de la implementación de la solución planteada, se consideran los siguientes entregables:

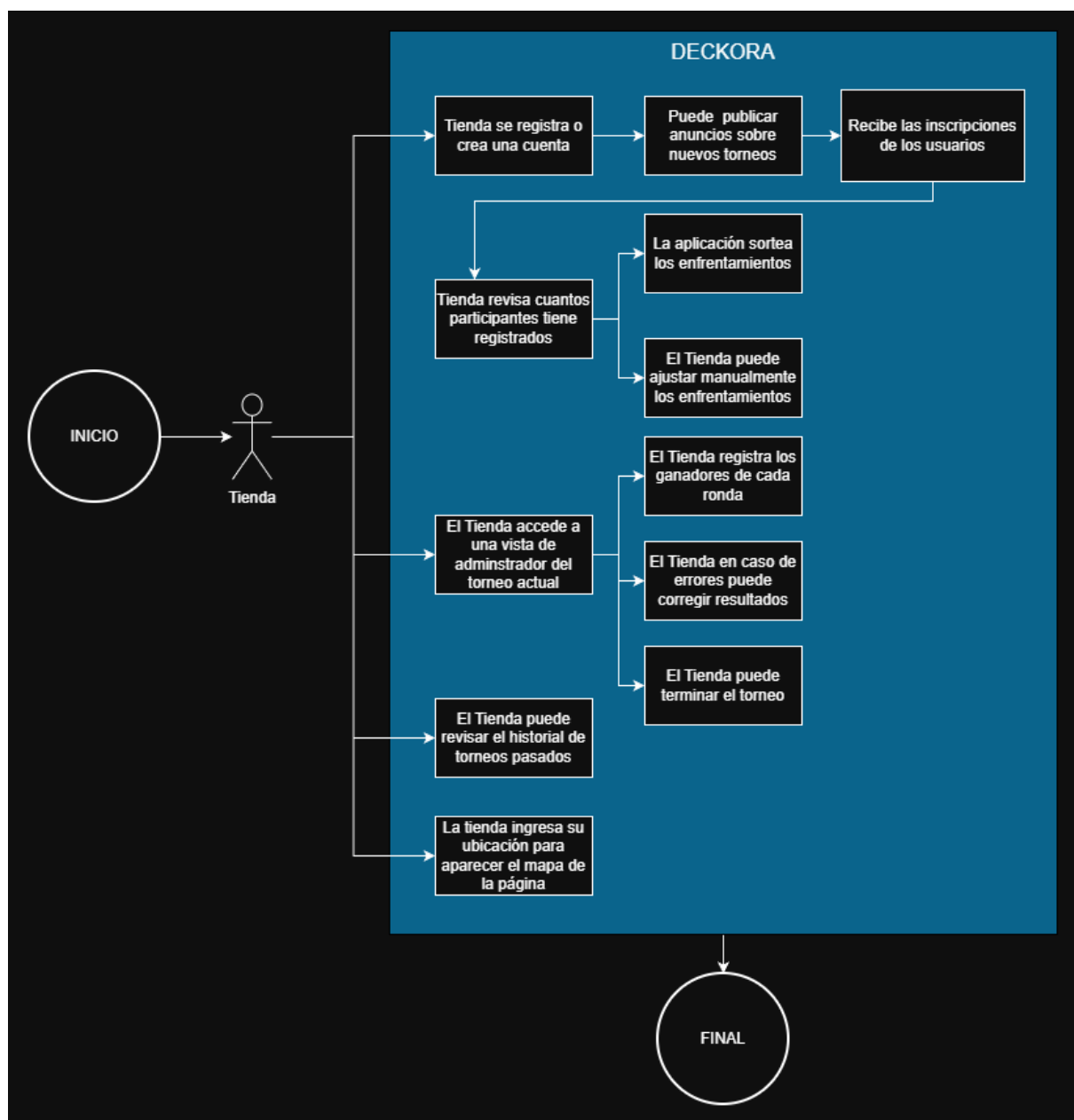
- **Prototipo funcional de la plataforma web:** Aplicación desplegada en Vercel y Render que permita ejecutar el flujo completo de usuario y organizador
- **Código fuente del sistema desarrollado:** Acceso al repositorio con la implementación del frontend (React) y backend (Node.js/Express)
- **Documentación técnica del sistema:** Manual que detalle la arquitectura cloud, el esquema de la base de datos en Supabase y el funcionamiento de la integración con IA.
- **Evidencias de las pruebas realizadas:** Reporte detallando las pruebas de usuario y de sistema que aseguren la estabilidad de la plataforma.

Para el desarrollo de la plataforma nos regimos por estándares que aseguran la protección tanto de la información del usuario como la estabilidad del sistema bajo una alta demanda. Para esto, tomamos las siguientes consideraciones:

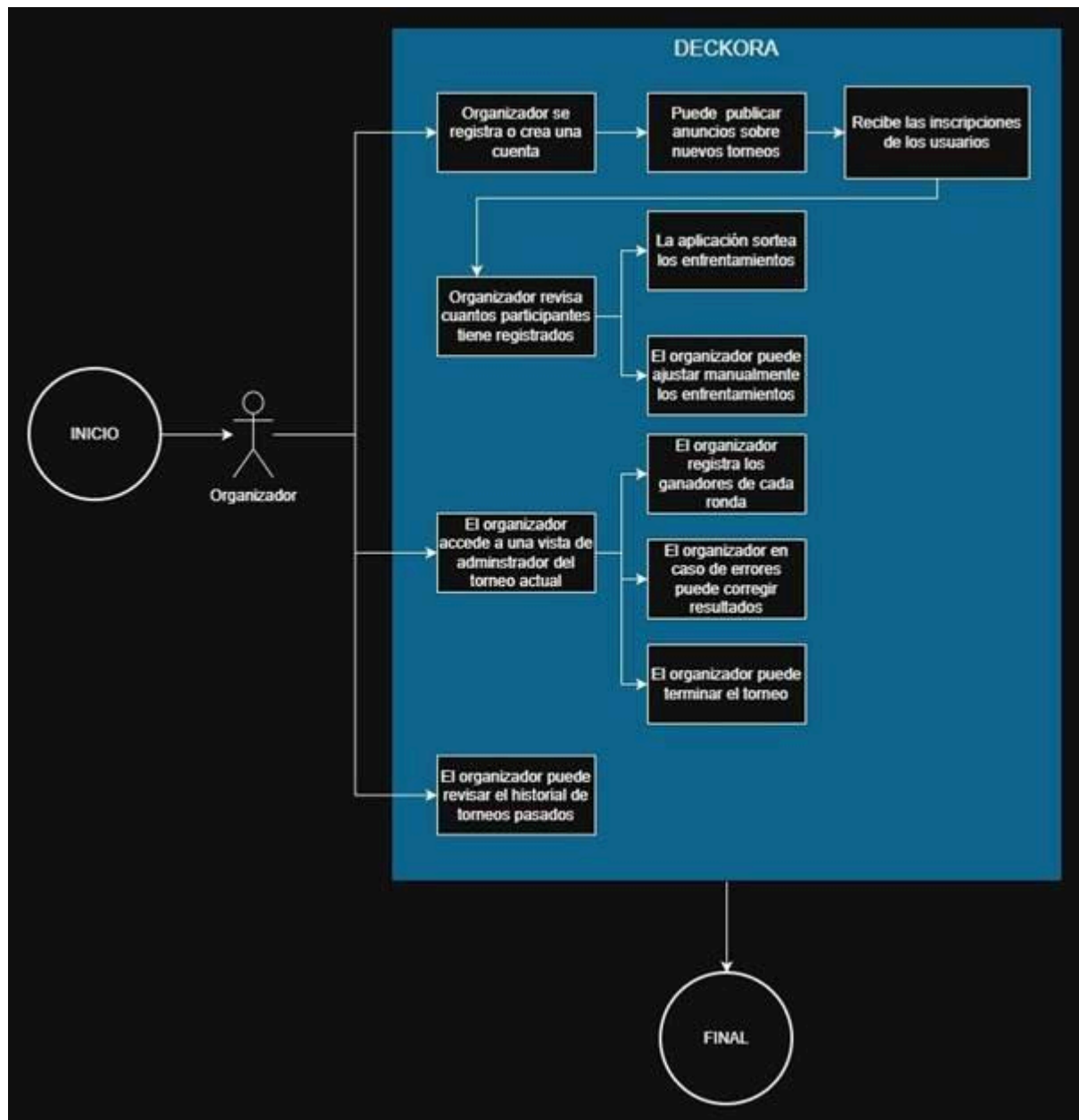
- **Autenticación y autorización robusta:** Controlamos el acceso mediante Supabase Auth, utilizando tokens de seguridad para garantizar que solo los usuarios propietarios puedan editar sus perfiles o mazos
- **Integridad y validación de los datos:** Se implementan reglas tanto en el frontend como en el backend para asegurar que los resultados de torneos sean consistentes y la creación de mazos cumpla con los requisitos técnicos de las APIs externas
- **Respaldo y disponibilidad:** Al utilizar una base de datos en la nube (PostgreSQL), se cuenta con mecanismos de respaldo automático y alta disponibilidad, minimizando el riesgo de pérdida de información de los jugadores.



5. Diagrama To-Be perfil jugador



6. Diagrama To-Be perfil tienda



7. Diagrama To-Be perfil organizador

9. Alcance, supuestos y restricciones

Dentro del proyecto de Decora, contemplamos el desarrollo de una plataforma web orientada a la gestión de herramientas e información dentro de los entornos de TCG. A continuación, delimitamos el alcance del sistema, junto con los supuestos y restricciones tomadas a consideración para su desarrollo.

9.1 Alcance

Definimos que se desarrollará y que incluirá la solución final, estableciendo los límites del sistema. El objetivo es dejar claro cuáles funciones, características y procesos formarán parte del proyecto final, así como cuales no estarán dentro del mismo.

Incluye:

- Registro e inicio de sesión de usuarios
- Gestión de perfiles dentro de la plataforma (permitir borrar un usuario, editar correo o contraseña)
- Creación, edición y gestión de mazos
- Creacion, edicion y gestion de carpetas
- Visualización de estadísticas básicas asociadas al rendimiento de jugadores
- Inscripción de usuarios y registro de resultados de torneos o eventos
- Visualización de torneos disponibles
- Publicación de eventos por parte de tiendas y organizaciones
- Visualizacion de informacion de tiendas y eventos dentro de la plataforma
- Inclusión de asistente de IA

No incluye:

- Desarrollo de una aplicación móvil
- Implementación de sistemas de pago o monetización dentro de la plataforma
- Desarrollo de funcionalidades avanzadas de inteligencia artificial
- Gestión de inventario físico de cartas en tiendas
- Soporte para juegos TCG fuera de Magic

9.2 Entregables

Como resultado del desarrollo, se consideran los siguientes entregables:

- Prototipo funcional de la plataforma web
- Código fuente del sistema desarrollado
- Documentación técnica del sistema
- Evidencias de las pruebas realizadas

9.3 Supuestos

Como Supuestos, consideramos a condiciones que asumimos como verdaderas, sin necesidad de comprobación durante el desarrollo. Estos representan factores externos que se dan por hecho, como el comportamiento de los usuarios, disponibilidad de recursos o acceso a información.

- Se asume que los usuarios cuentan con acceso a internet y dispositivos compatibles con plataformas web.
- Se asume que los datos necesarios (mazos, torneos y resultados) podrán ser ingresados por los usuarios.
- Se asume que los datos necesarios para mazos, torneos y resultados estarán disponibles.
- Se asume que los organizadores y tiendas estarán dispuestos a adoptar la plataforma.
- Se asume que el usuario está dispuesto a registrarse dentro de la plataforma.

9.4 Restricciones

Corresponden a las limitaciones o condiciones que afectan el desarrollo y ejecución del proyecto. Estas pueden estar relacionadas con el tiempo disponible, tecnología, el acceso a información o presupuesto. Definen el contexto en el cual se trabaja.

- Tiempo limitado al periodo académico.
- Implementación limitada a el TCG de Magic.

10. Metodología de desarrollo y gestión del proyecto

10.1 Metodología utilizada

Para Deckora se utilizó una metodología híbrida que combina tres enfoques complementarios, cada uno aplicado en la fase del proyecto donde aporta más valor:

- **Enfoque secuencial en la fase de diseño:** durante las primeras etapas del proyecto se trabajó de forma ordenada y paso a paso, definiendo los requisitos funcionales y no funcionales, el modelo de datos, la arquitectura general y los flujos principales antes de escribir código. Esta fase se asemeja a un enfoque tradicional porque las decisiones de diseño (modelo de datos, separación en módulos, elección del stack) condicionan todo el desarrollo posterior y conviene cerrarlas antes de empezar a programar.
- **Scrum en la fase de desarrollo:** una vez definido el diseño, se pasó a iteraciones cortas con planificación, ejecución, revisión y ajuste continuos. Cada iteración entrega funcionalidades completas y verificables, y las decisiones se revisan al cierre de cada sprint para incorporar aprendizajes al siguiente.
- **Extreme Programming (XP) en las tareas de QA:** en las tareas que requerían validación cuidadosa, uno de los integrantes desarrollaba y probaba la funcionalidad mientras el otro hacía backseating: observando, revisando el código en vivo y detectando errores o casos borde que el desarrollador principal podía pasar por alto. Esta dinámica de programación en pareja redujo el tiempo dedicado a corrección de errores posteriores y mejoró la calidad del código entregado.

Deckora es un proyecto con alcance grande para un equipo de dos personas y plazos acotados. El enfoque secuencial inicial dio una base sólida para no improvisar arquitectura sobre la marcha; Scrum permitió adaptar el plan cuando aparecieron desviaciones; y XP fue clave para mantener calidad sin necesidad de un rol dedicado de QA. Al trabajar semana a semana se nos permitió:

- **Planificación:** al inicio de cada iteración se seleccionaban las tarjetas a abordar desde el tablero Trello, priorizando funcionalidades bloqueantes (autenticación, modelo de datos, módulos centrales) antes de las complementarias (estadísticas, integraciones de notificación).
- **Seguimiento:** el avance se reflejaba en el tablero moviendo tarjetas entre columnas (Por hacer → En progreso → En revisión → Hecho). Las decisiones técnicas relevantes se discutían directamente entre ambos integrantes.
- **Control:** el flujo de Pull Requests obliga a que toda funcionalidad pase por revisión del otro integrante antes de integrarse, lo que actúa como control de calidad continuo. Si una funcionalidad no cumplía la Definition of Done, la PR no se aprobaba.

10.2 Roles y responsabilidades

El equipo está compuesto por dos integrantes que trabajan de forma colaborativa sin una división rígida de roles:

Integrante	Responsabilidades
Vicente Verdaguer	Desarrollo full-stack (frontend, backend), diseño de base de datos, QA, despliegue, documentación
Anais Palma	Desarrollo full-stack (frontend, backend), diseño de base de datos, QA, despliegue, documentación

Ambos participaron en todas las áreas del proyecto (frontend, backend, integración con servicios externos, pruebas y documentación). La asignación específica de cada tarea se acordaba al momento de planificar la iteración según disponibilidad y contexto previo.

10.3 Definition of Done (DoD)

Una funcionalidad se considera “Hecha” únicamente cuando cumple todos los siguientes criterios:

Implementación completa: el código cubre todos los casos definidos en la historia de usuario, incluidos los caminos de error.

Validación de entrada: en backend, todo endpoint que recibe datos tiene su schema de validación con Zod. En frontend, los formularios se validan antes de enviar.

Manejo de errores: los errores se capturan y se traducen a respuestas HTTP con código apropiado (backend) o a mensajes de UI comprensibles (frontend).

Control de acceso: Los endpoints que requieren autenticación usan el middleware auth, y los que requieren rol específico usan requirePerfil.

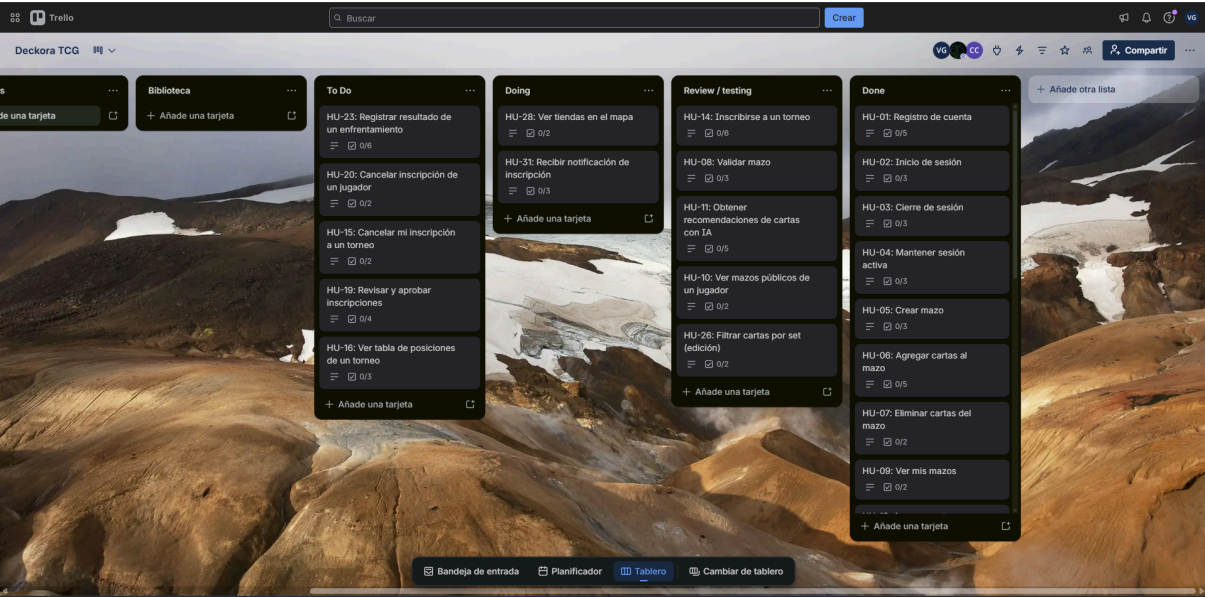
Convenciones de código respetadas: PascalCase para modelos y componentes, prefijo use en hooks, dominio.tipo.js en archivos de módulos backend, sin warnings de ESLint.

Probado en local: el desarrollador probó manualmente los caminos felices y al menos un caso de error.

Revisión por pares: la funcionalidad pasó por una Pull Request revisada y aprobada por el otro integrante antes de hacer merge a dev.

Integración exitosa: tras el merge a dev, no se rompió ninguna funcionalidad existente.

Documentación mínima: si la funcionalidad introduce un nuevo endpoint, módulo o decisión arquitectónica relevante, queda reflejada en la documentación técnica o funcional.



8. Tareas en Trello

Sprint / Hito	Fecha estimada	Entregables principales
Sprint 1 — Definición y análisis del proyecto	13-04-2026 / 26/04/2026	Modelo de datos, arquitectura, repositorios creados
Sprint 2 — Módulos centrales	27-04-2026 /10-05-2026	CRUD de mazos, biblioteca de cartas, gestión básica de torneos
Sprint 3 — Desarrollo frontend y backend	11/05/2026 – 24/05/2026	Aplicación funcional.
Sprint 4 — Testing	25/05/2026 – 07/06/2026	Resultados de casos pruebas, documentación de bugs

11. Documentos y diagramas de diseño de la solución

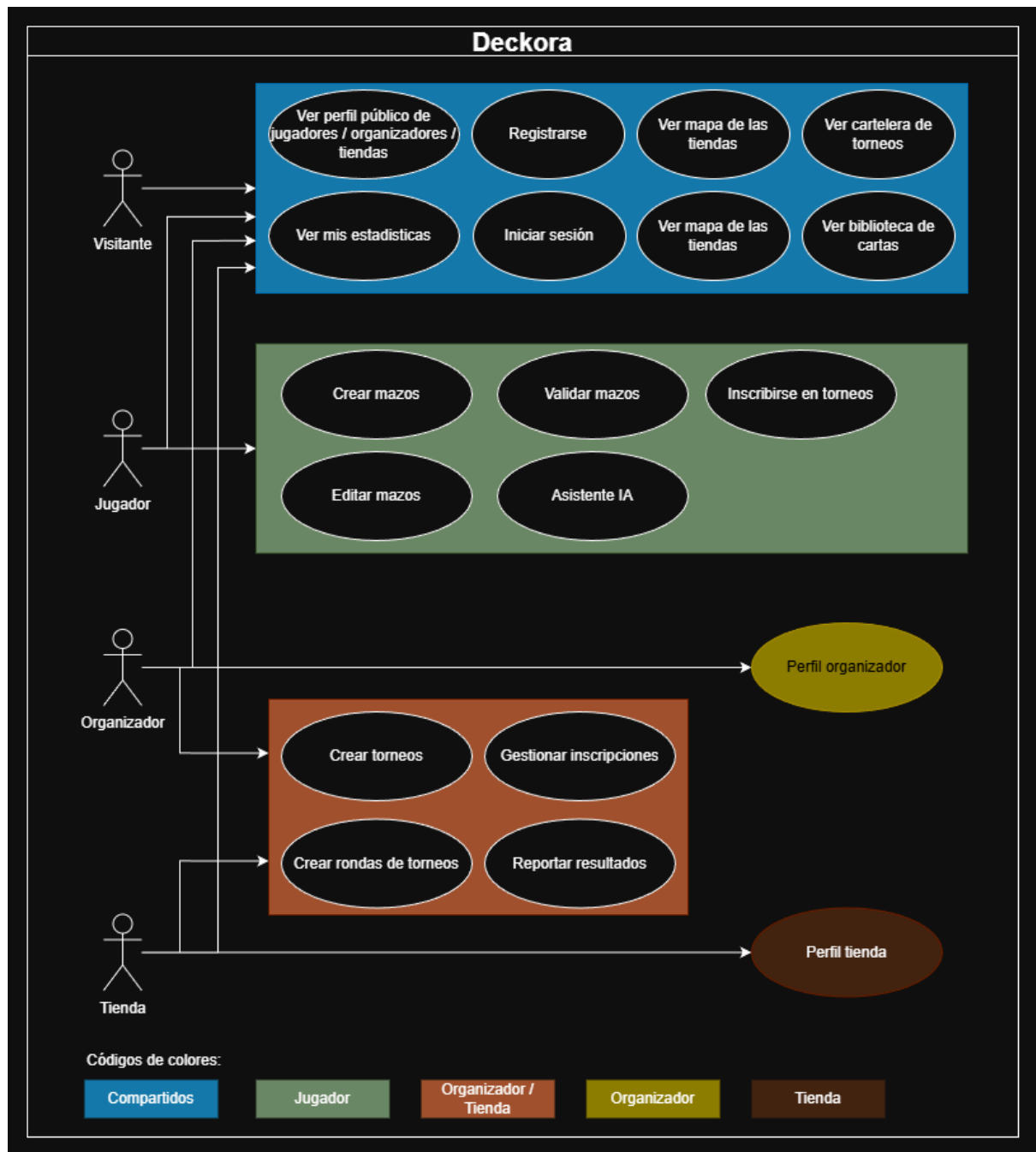
Para Deckora se elaboraron cinco diagramas que estructuran el diseño de la solución desde distintas perspectivas: los actores y sus interacciones, los procesos del negocio, la apariencia visual del sistema, la estructura de los datos y la arquitectura técnica.

11.1 Casos de uso / Historias de usuario

Propósito: representar qué puede hacer cada tipo de usuario dentro del sistema. Es el mapa de funcionalidades visto desde la perspectiva de los actores.

Qué representa: los tres roles definidos (Jugador, Organizador, Tienda) y un actor anónimo (Visitante), junto con las acciones que cada uno puede realizar. Por ejemplo, el Jugador puede Crear mazo, Usar Asistente IA, Inscribirse a torneo y Ver estadísticas; el Organizador puede Crear torneo, Aprobar inscripciones, Gestionar rondas y Reportar resultados; el Visitante puede Ver cartelera de torneos y Explorar biblioteca de cartas.

Relación con el problema y los objetivos: el problema central que aborda Deckora es la falta de una plataforma unificada para la comunidad de Magic: The Gathering que conecte jugadores con organizadores y tiendas. Este diagrama hace explícito que cada tipo de actor tiene su propio conjunto de capacidades dentro de un mismo ecosistema, justificando la decisión arquitectónica de separar perfiles extendidos (jugadores, organizadores, tiendas) por encima de la tabla común usuarios.



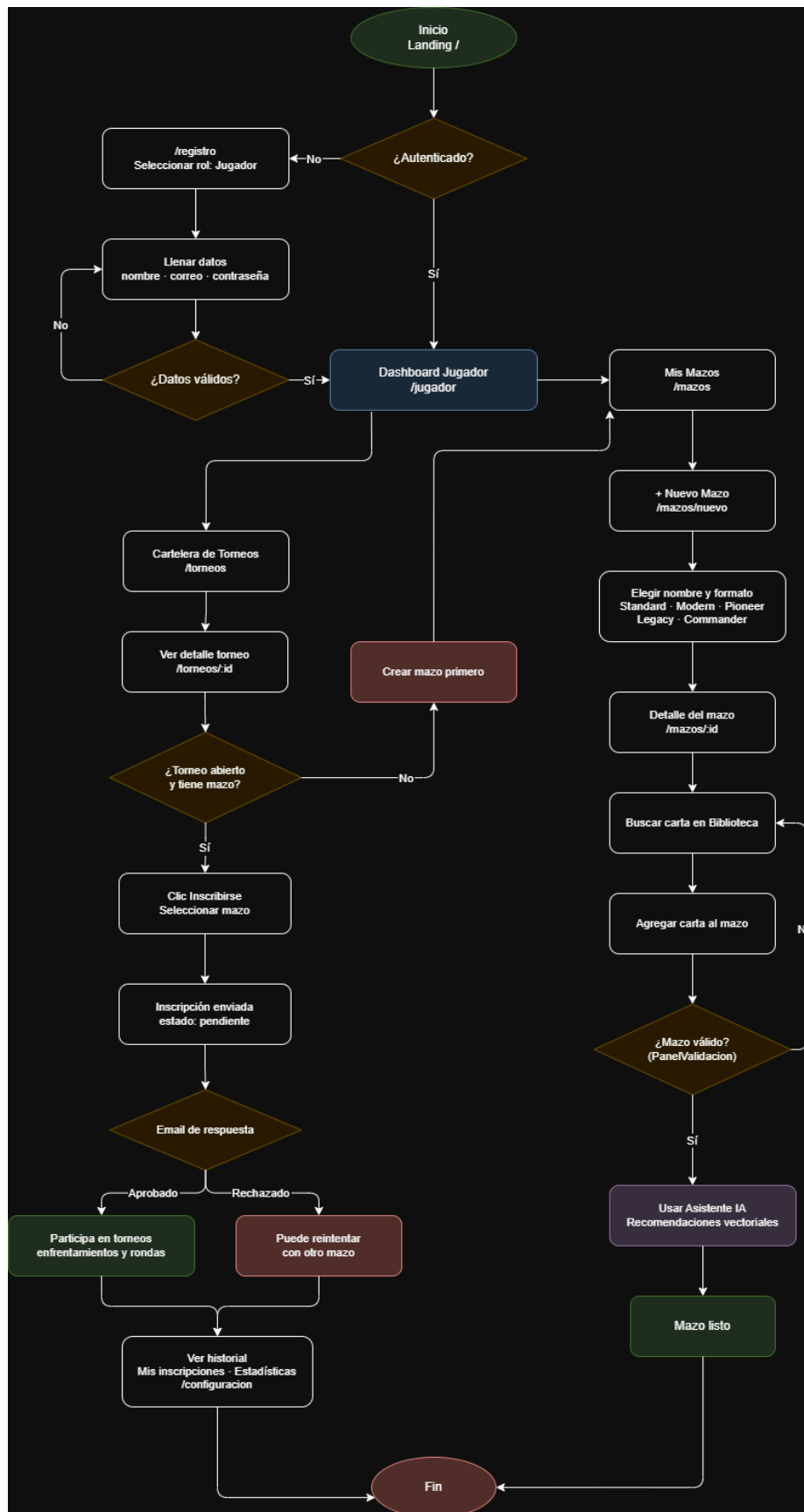
9. Diagrama de casos de uso

11.2 Diagrama de flujo de usuario (User Flow)

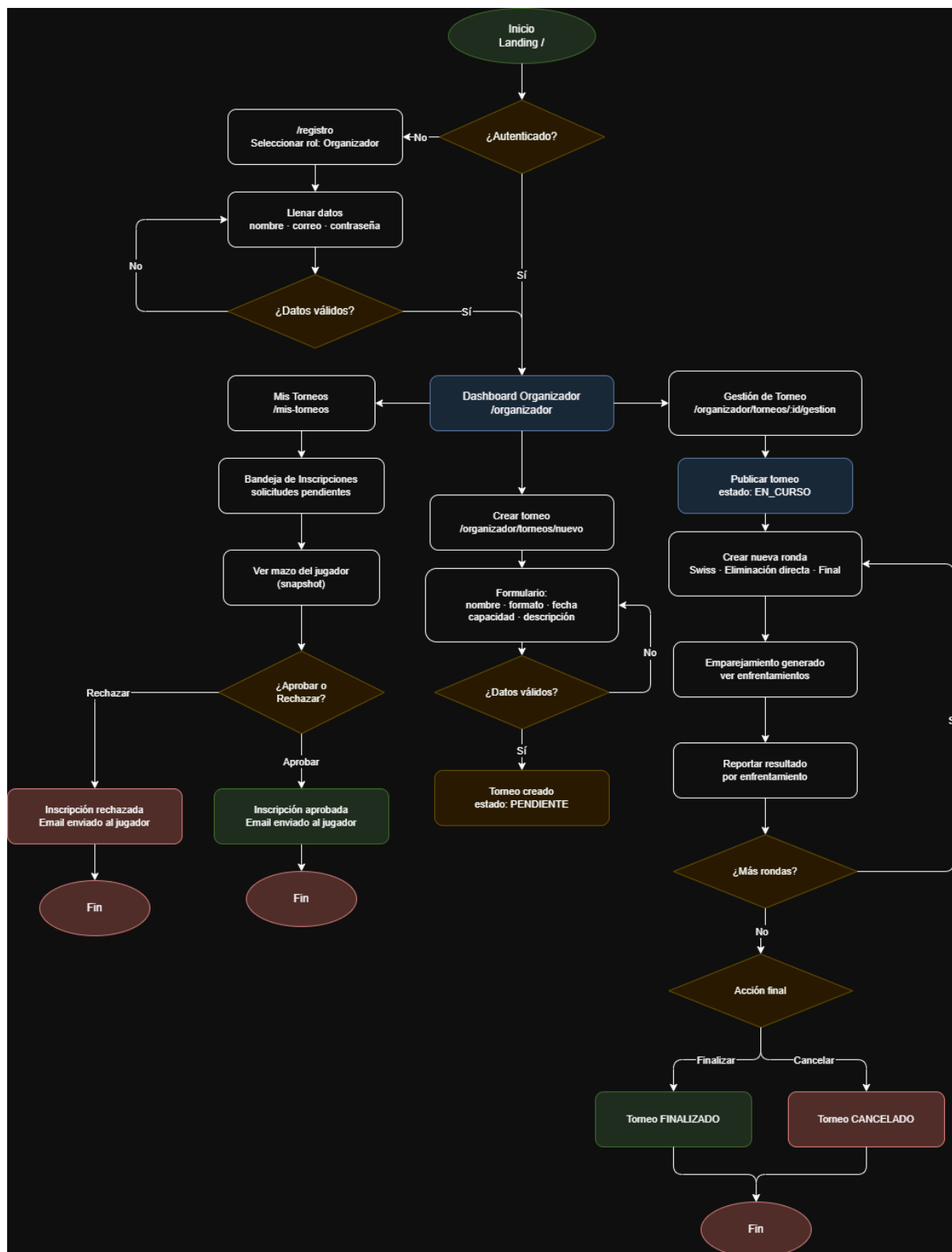
Propósito: mostrar cómo navega un usuario a través del sistema para completar tareas concretas, paso a paso.

- **Flujo del Jugador:** desde la landing pública → registro como jugador → crear mazo → usar Asistente IA → inscribirse a torneo → ver historial.
- **Flujo del Organizador:** registro como organizador → crear torneo → recibir inscripciones → aprobar/rechazar → iniciar rondas → reportar resultados → cerrar torneo.

Relación con el problema y los objetivos: este diagrama valida que los caminos críticos del sistema están diseñados sin cuellos de botella, decisiones ambiguas o pantallas faltantes. Confirma que un jugador nuevo puede ir desde el primer ingreso hasta inscribirse a un torneo sin pasos innecesarios.



10. Diagrama de flujo de perfil jugador



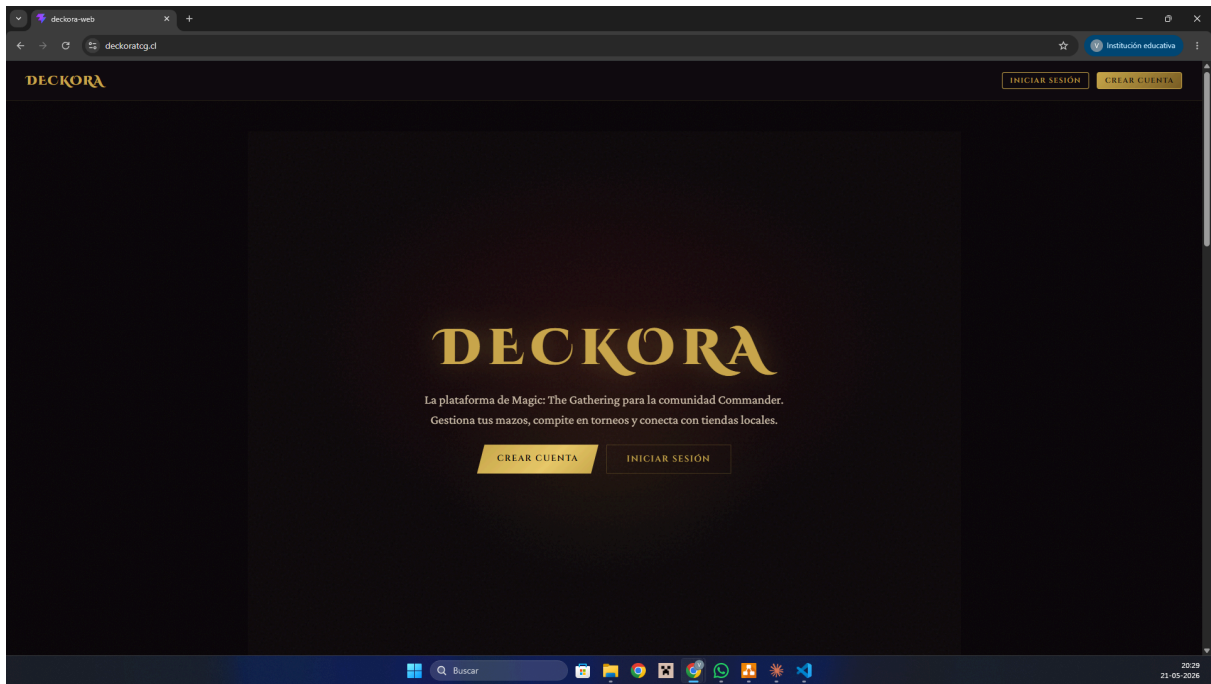
11. Diagrama de flujo de perfil organizador/tienda

11.3 Wireframes / Mockups de pantallas principales

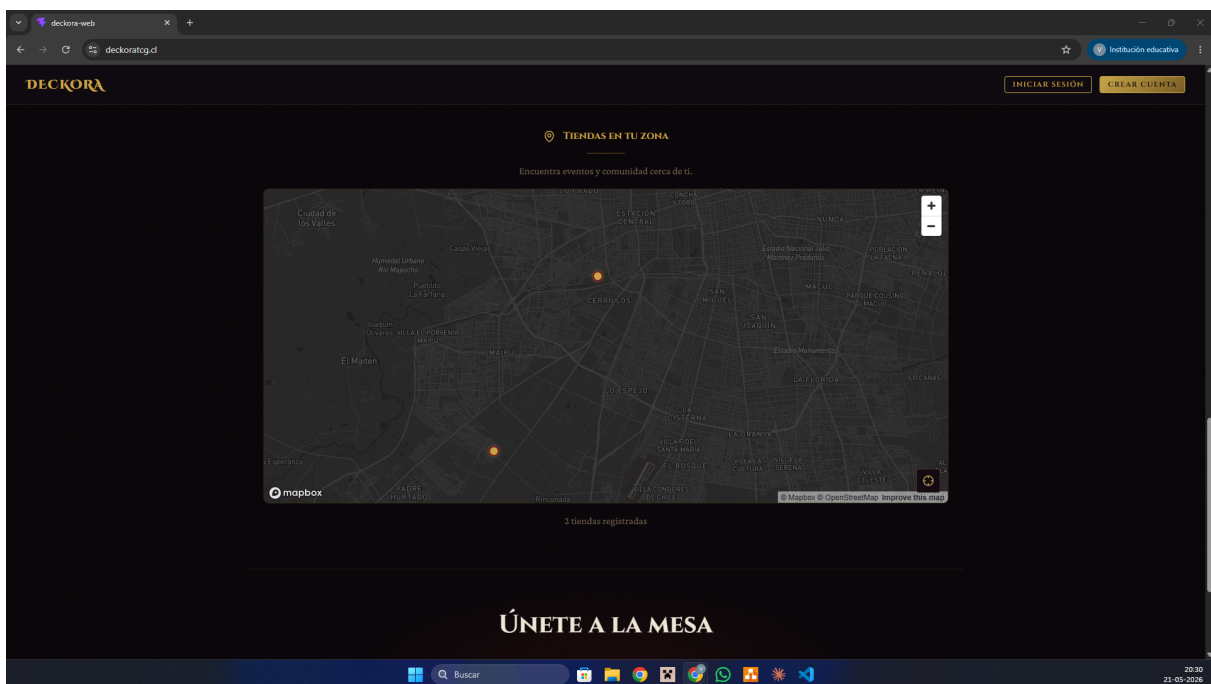
Propósito: mostrar la disposición visual de las pantallas clave del sistema, antes de entrar al detalle de estilos finales. Mockups realizados:

- Landing pública con cartelera de torneos y mapa de tiendas
- Detalle de mazo con el Asistente IA en pantalla
- Panel de gestión de torneo del organizador (rondas y emparejamientos)
- Vista de detalle de torneo desde la perspectiva de un participante inscrito
- Dashboard del jugador con estadísticas y mazos

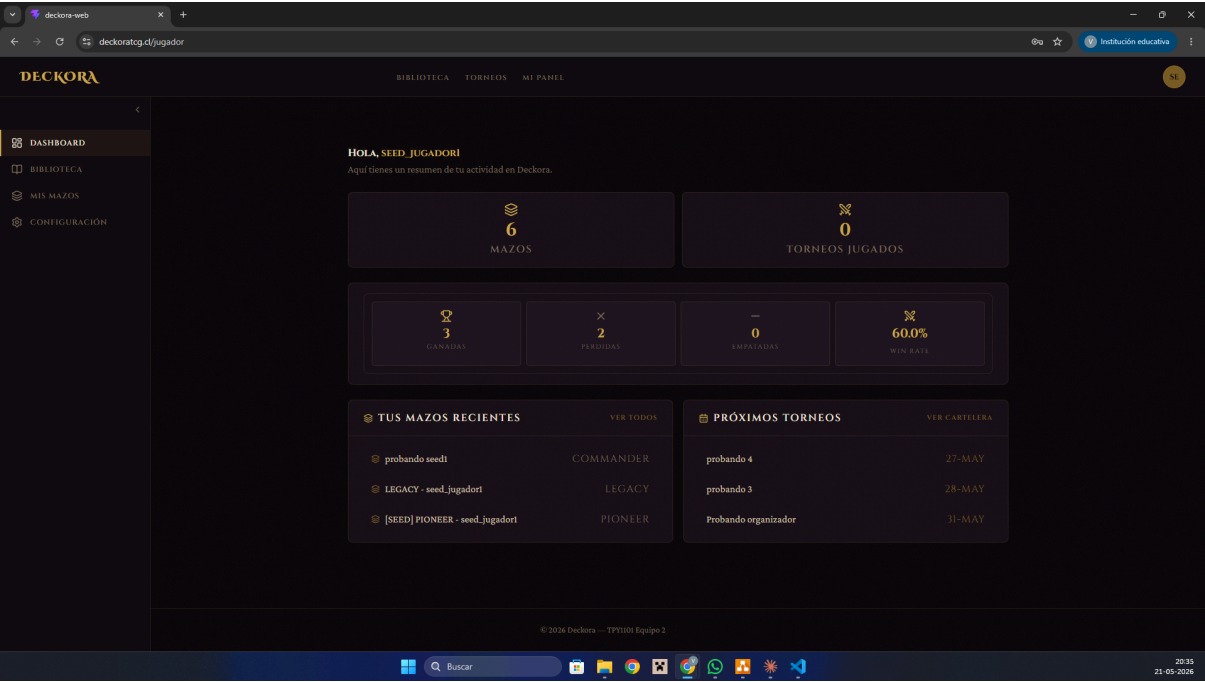
Relación con el problema y los objetivos: los wireframes garantizan que la información crítica esté visible en el lugar correcto. Por ejemplo, que en el panel de gestión de rondas el organizador vea simultáneamente las mesas, los resultados pendientes y la opción de cerrar la ronda, sin tener que navegar a varias pantallas.



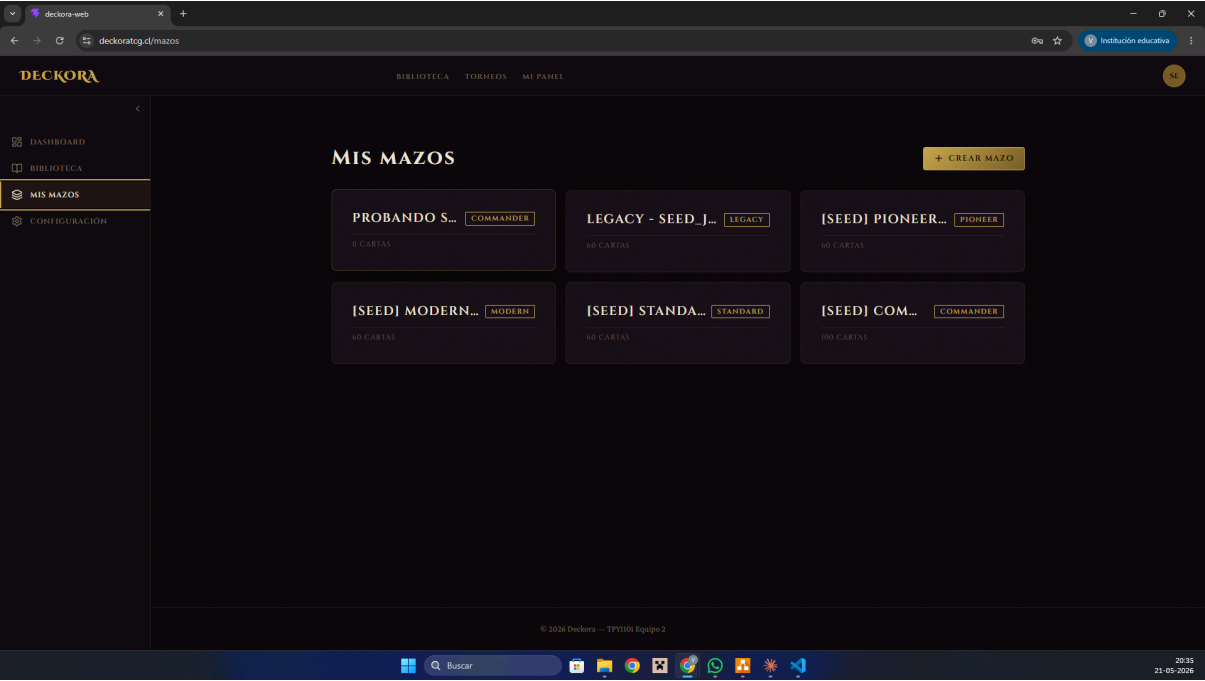
12. Landing page: hero



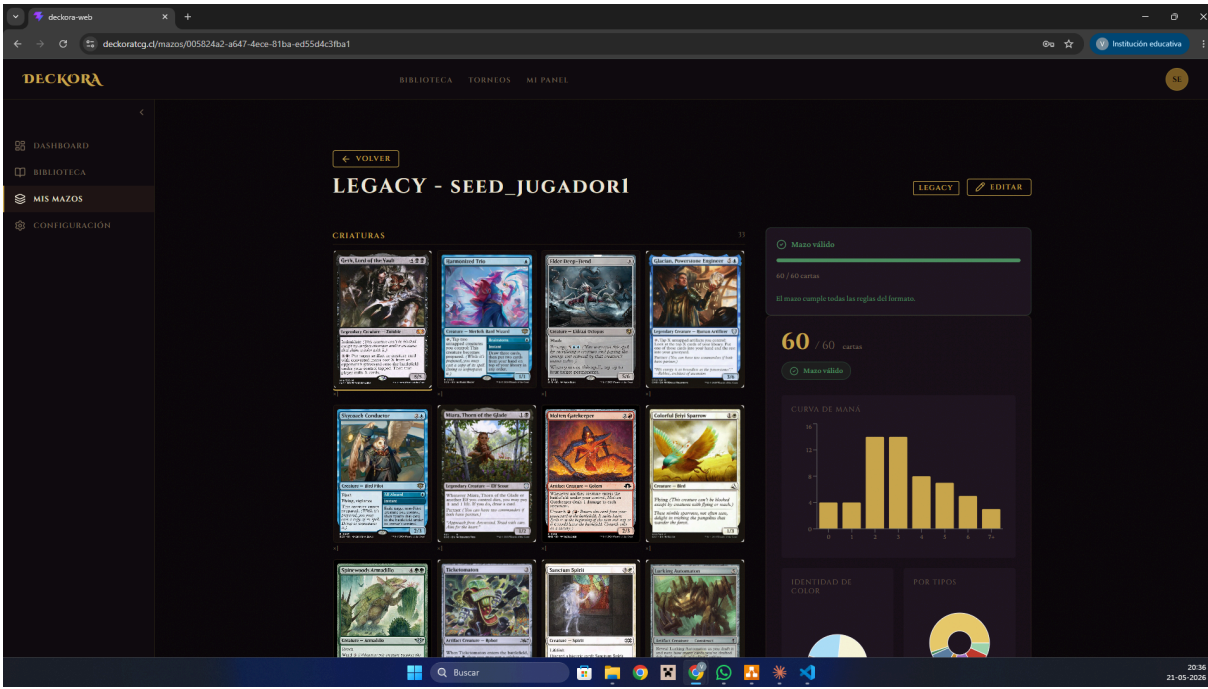
13. Landing page: mapa



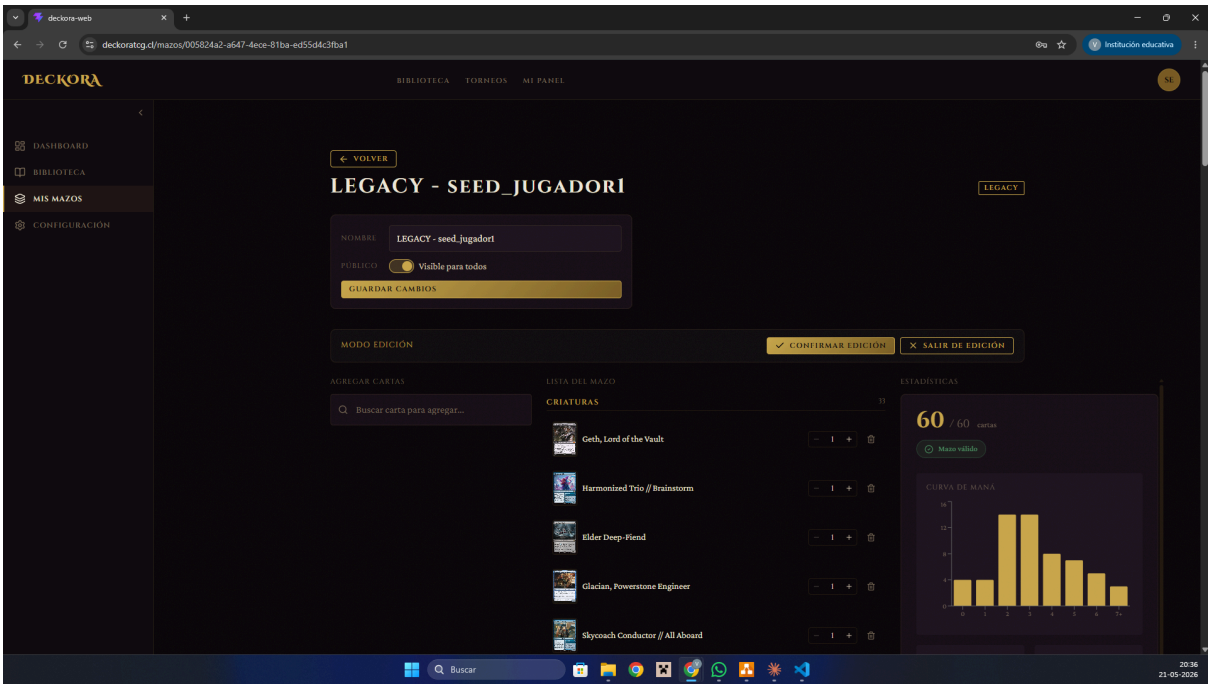
14. Perfil de jugador: dashboard



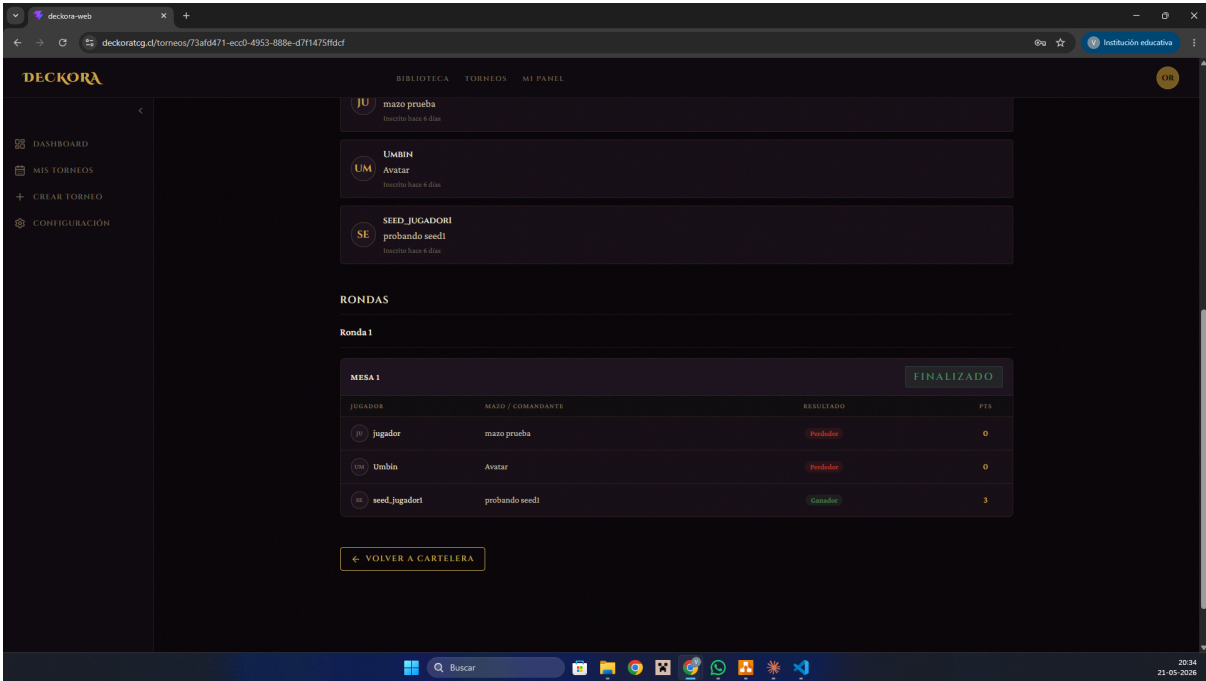
15. Perfil jugador: mis mazos



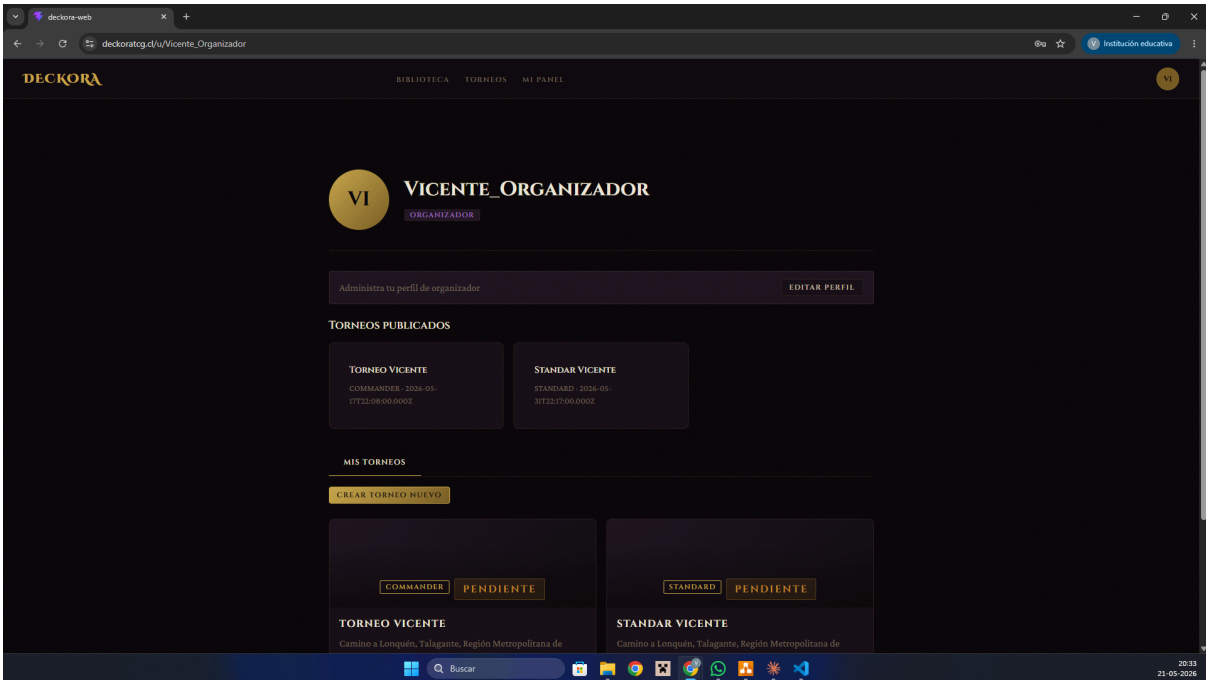
16. Perfil jugador: detalle de mazo



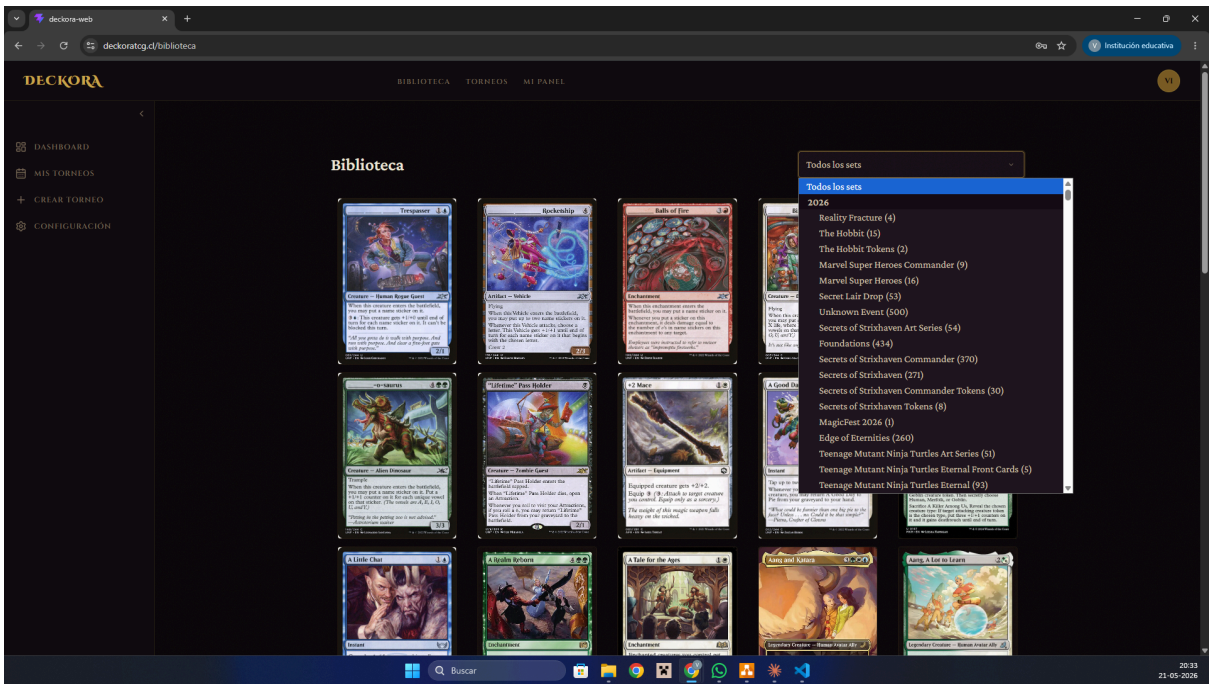
17. Perfil jugador: edición de mazo



18. Perfil organizador: gestión de torneos



19. Perfil organizador: vista de perfil



20. Biblioteca: filtro por edición

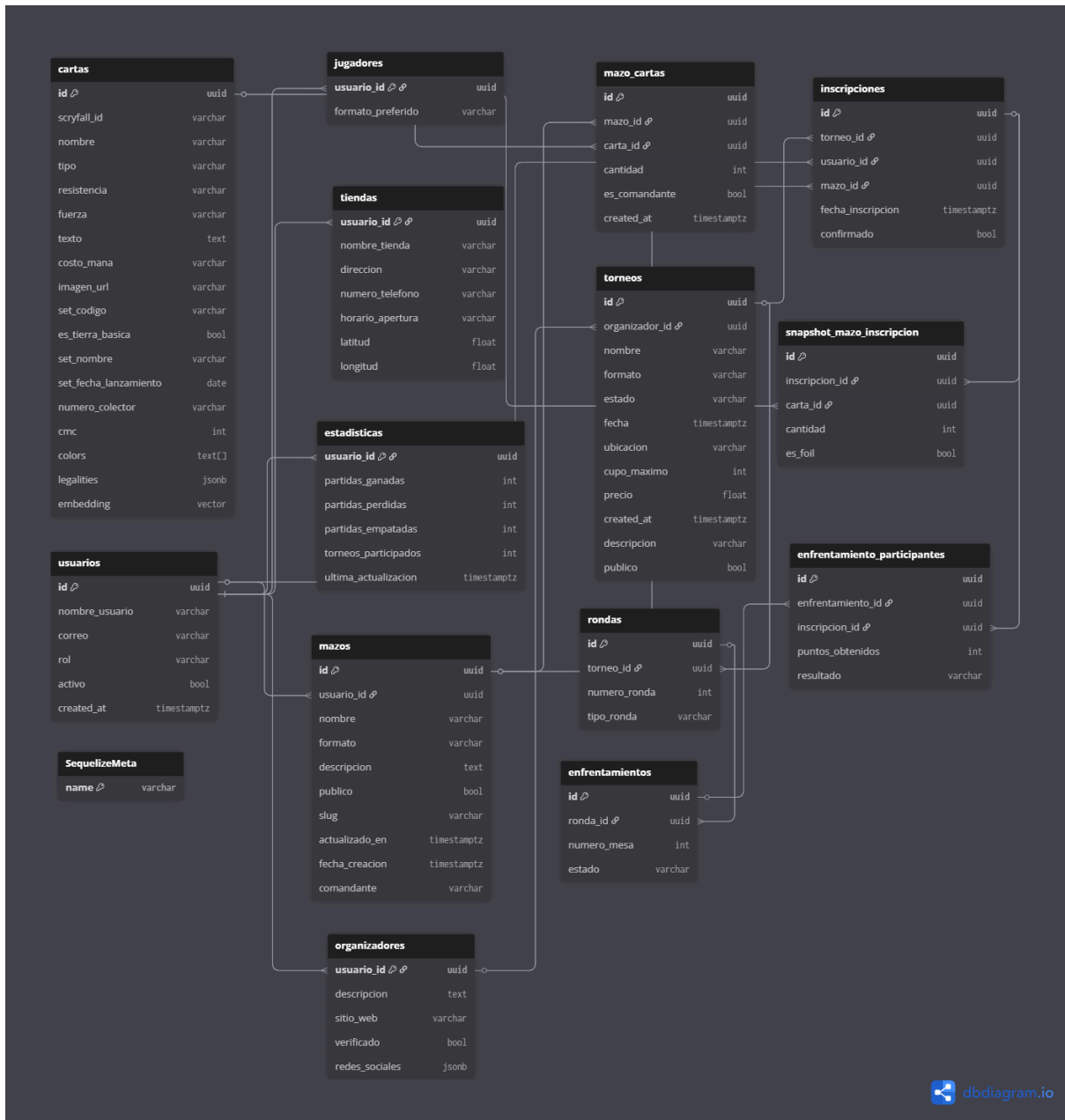
11.4 Modelo de datos (MER / DER)

Propósito: representar la estructura de la base de datos y las relaciones entre entidades.

Qué representa: las entidades principales del sistema y sus relaciones. A partir de la documentación, las entidades núcleo son:

- **usuarios** (entidad base con id, nombre_usuario, correo, rol)
- **jugadores, organizadores, tiendas** (perfiles extendidos, 1:1 con usuarios)
- **estadísticas** (1:1 con jugadores)
- **mazos** (N:1 con jugadores)
- **cartas** (catálogo de ~32.500 cartas con campo embedding vector(768))
- **mazo_cartas** (tabla puente N:M entre mazos y cartas)
- **torneos** (N:1 con organizadores o tiendas)
- **inscripciones** (N:M entre jugadores y torneos)
- **rondas** (N:1 con torneos)
- **enfrentamientos** (N:1 con rondas)
- **enfrentamiento_participantes** (N:M entre enfrentamientos e inscripciones)

Relación con el problema y los objetivos: el modelo separa información común (usuarios) de información específica por rol (perfiles extendidos), lo que evita columnas nulas y permite que cada rol tenga campos propios sin afectar a los demás. La tabla puente mazo_cartas permite que la misma carta aparezca en miles de mazos sin duplicación. El campo embedding en cartas es lo que habilita el motor de recomendación basado en IA.



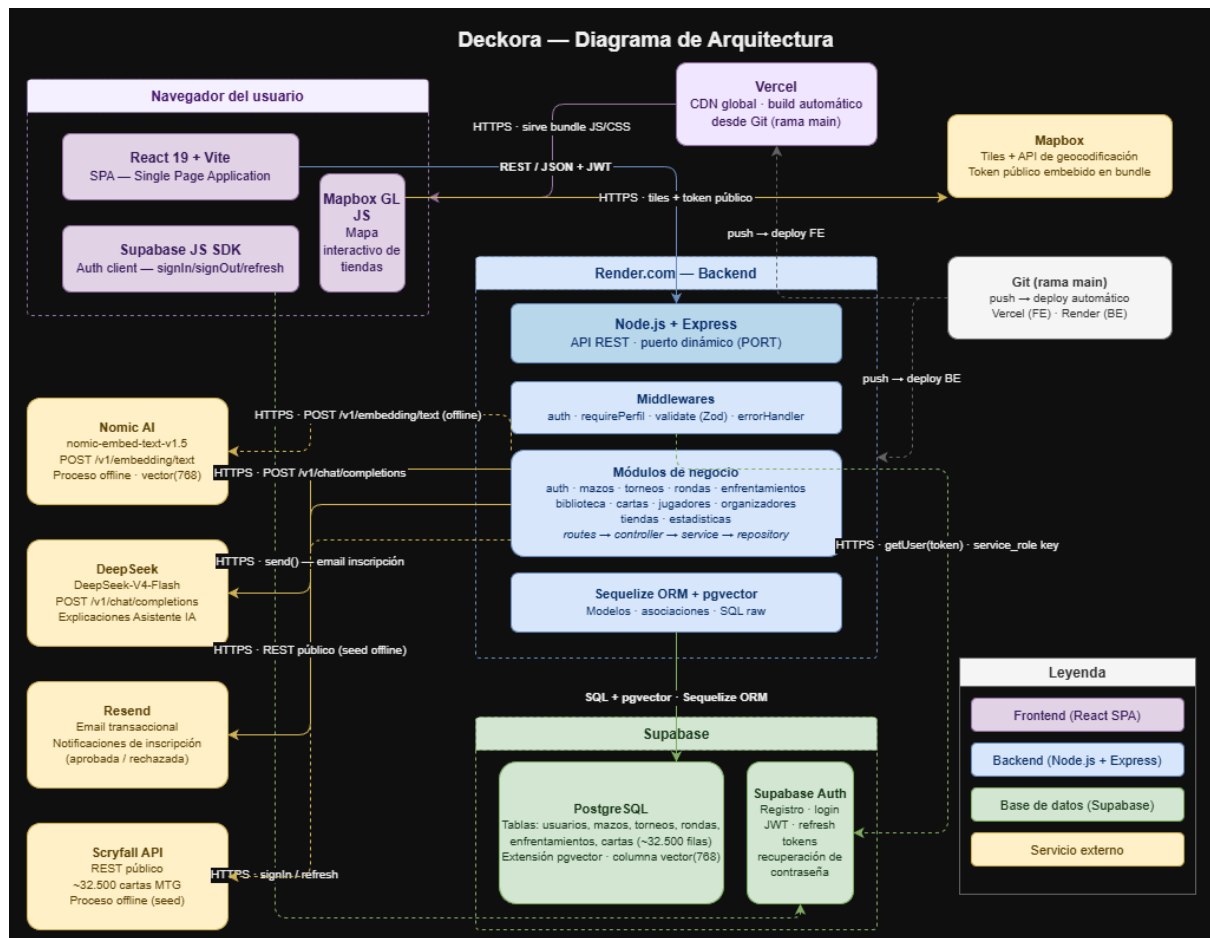
21. Modelo de base de datos

11.5 Arquitectura de alto nivel

Propósito: mostrar los componentes técnicos del sistema y cómo se comunican entre sí.

Qué representa: los tres bloques principales (Frontend en Vercel, Backend en Render, Base de datos en Supabase) más los servicios externos (Nomic AI, DeepSeek, Mapbox, Resend, Supabase Auth), con flechas indicando el sentido de comunicación y el protocolo o tipo de dato que viaja por cada conexión.

Relación con el problema y los objetivos: la elección de arquitectura en tres capas desacopladas conectadas por HTTP permite escalar, mantener y desplegar cada capa de forma independiente. La delegación de la autenticación a Supabase Auth y de los embeddings a Nomic AI evita reimplementar servicios complejos y críticos en seguridad.



22. Diagrama de arquitectura

12. Arquitectura de software y patrones de diseño

12.1 Arquitectura utilizada

Deckora implementa una arquitectura de tres capas desacopladas comunicadas por HTTP, donde cada capa está desplegada en infraestructura independiente y se comunica con la siguiente mediante un protocolo definido:

Capa de Presentación — React + Vite (SPA), desplegado en Vercel
↓ REST/JSON + JWT
Capa de Negocio — Node.js + Express, desplegado en Render
↓ Sequelize ORM + pgvector
Capa de Datos — PostgreSQL (Supabase) — ~32.500 cartas con vector(768)

Justificación: esta separación es adecuada para Deckora porque permite que el frontend se construya y despliegue de forma totalmente independiente del backend (Vercel construye y publica al hacer push a main del repo web, sin tocar el servidor), y porque la base de datos puede escalarse o reemplazarse sin afectar la lógica de negocio. Además, la capa de presentación al ser una SPA distribuida por CDN ofrece tiempos de carga bajos a nivel global, mientras que el backend puede dedicarse exclusivamente a la lógica.

12.2 Patrones de diseño aplicados

Deckora aplica varios patrones complementarios, cada uno en la capa donde aporta más valor:

Strategy (Estrategia)

Aplicado en dos lugares del backend:

- **Validación de formatos de mazo (src/modules/mazos/estrategias/)**: cada formato de Magic (Commander, Standard, Modern, Pioneer, Legacy) tiene su propio archivo de validación con sus reglas particulares (cantidad de cartas, restricciones de comandante, banlist, etc.). El servicio selecciona la estrategia correcta en tiempo de ejecución según el formato del mazo. Agregar un nuevo formato no requiere modificar código existente: solo crear un archivo y registrarlo en el índice.
- **Emparejamiento de rondas (src/modules/rondas/emparejadores/)**: el mismo patrón se aplica al sistema de emparejamiento (swiss, eliminacion_directa, final), donde cada tipo de ronda tiene su algoritmo intercambiable.

Repository

Toda la interacción con la base de datos está encapsulada en archivos *.repository.js. Los servicios no usan Sequelize directamente: llaman a funciones del repositorio que devuelven objetos de dominio. Esto desacopla la lógica de negocio del ORM, permitiendo cambiar de Sequelize a otra librería (o incluso a SQL puro) sin tocar los servicios.

Layered Architecture (capas de responsabilidad)

Dentro del backend cada módulo sigue la misma jerarquía estricta: routes.js → controller.js → service.js → repository.js. El controller nunca llama al repository directamente; el repository nunca llama al service. Esta regla hace que cada capa se pueda testear de forma independiente y que el rol de cada archivo sea predecible para cualquier desarrollador que entre al proyecto.

Middleware Chain (Cadena de responsabilidad)

Las rutas Express encadenan middlewares que se ejecutan en orden: autenticación, verificación de rol, validación de schema y finalmente la lógica del controller. Si

cualquier eslabón falla, los siguientes nunca se ejecutan. Esto centraliza la seguridad en lugar de repetir validaciones en cada endpoint.

Facade (Fachada)

En el frontend, `src/services/api.js` oculta toda la complejidad de autenticación, inyección de tokens, manejo de errores HTTP y renovación de sesión. El resto del código solo invoca `apiGet`, `apiPost`, etc., sin preocuparse por esos detalles. Esta fachada también gestiona el reintento automático cuando el token expira.

Observer (Observador)

`AuthContext.jsx` se suscribe al evento `onAuthStateChange` de Supabase, lo que hace que la aplicación reaccione automáticamente a cambios de sesión en cualquier pestaña del navegador (por ejemplo, si el usuario cierra sesión en otra pestaña, todas las pestañas se actualizan).

Optimistic Update (Actualización optimista)

En la edición de mazos, cuando el jugador agrega o elimina una carta, la interfaz se actualiza inmediatamente sin esperar la respuesta del servidor. Si el servidor rechaza la operación, el estado se revierte. Esto hace que la UI se sienta instantánea incluso con latencia de red.

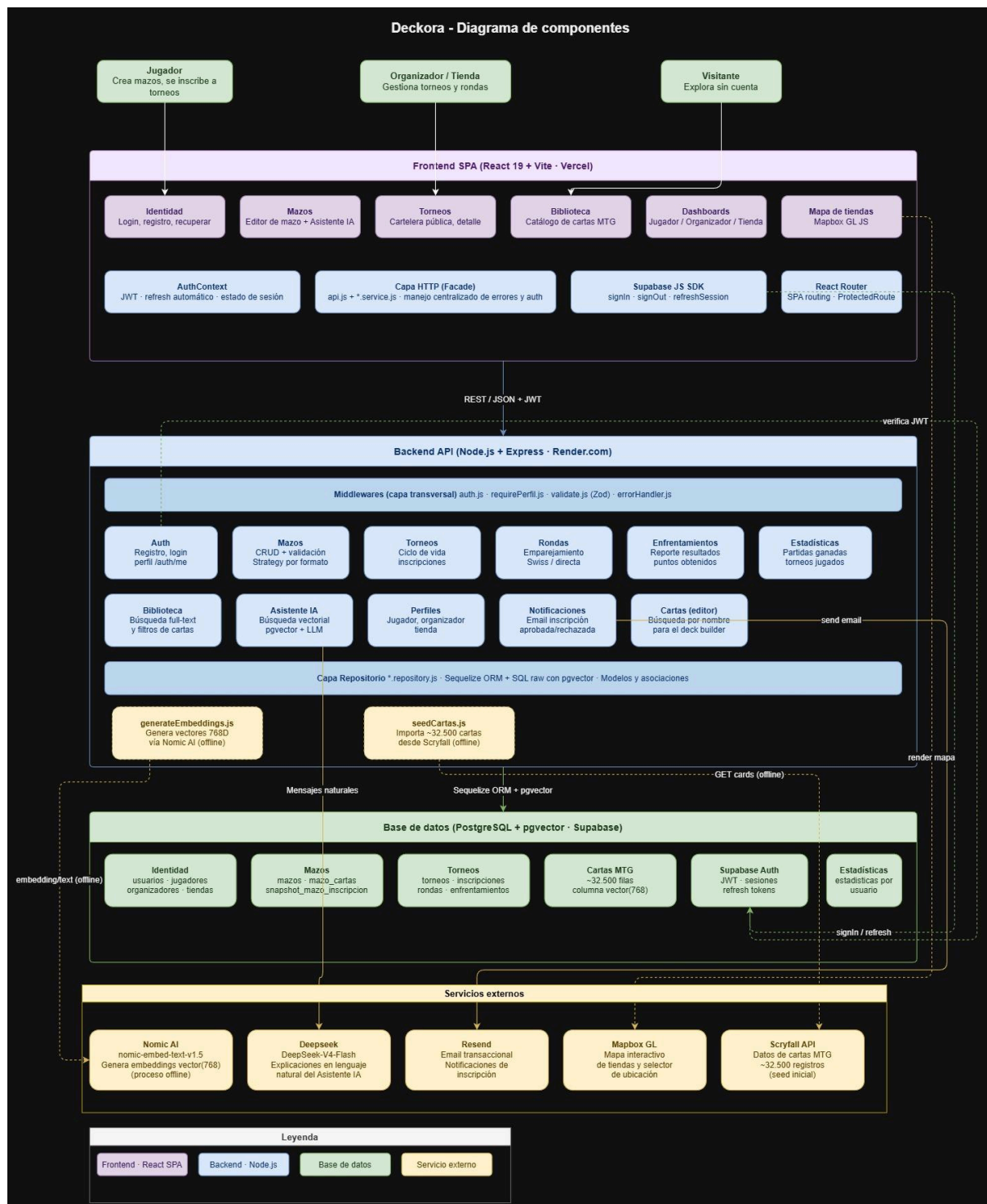
DTO (Data Transfer Object)

La función `serializarRonda()` convierte los objetos anidados que devuelve Sequelize (con relaciones en PascalCase y campos internos) en estructuras planas y limpias que el frontend espera consumir. Esto desacopla la forma de almacenamiento de la forma de presentación.

Retry con backoff exponencial

El script de generación de embeddings implementa reintentos automáticos con espera creciente (2s, 4s, 8s, 16s) cuando la API de Nomic AI devuelve `rate limit` (HTTP 429), lo que permite procesar las 32.500 cartas sin fallar por límites temporales.

12.3 Diagrama de componentes



23. Diagrama de componentes

12.4 Justificación de la arquitectura

La arquitectura en tres capas con módulos por dominio es adecuada para Deckora por cuatro razones concretas:

- **Tamaño del equipo (2 personas):** una arquitectura más compleja (microservicios, event-sourcing) habría introducido sobrecarga de coordinación innecesaria. Tres capas claras permiten que cualquiera de los dos integrantes trabaje en cualquier módulo sin pisar al otro.
- **Stack JavaScript end-to-end:** usar el mismo lenguaje en frontend y backend reduce el costo cognitivo de saltar entre capas.
- **Despliegue independiente:** Vercel y Render permiten que cada capa se actualice sin tocar la otra, lo que acelera los ciclos de iteración.
- **Escalabilidad incremental:** si en el futuro el módulo de IA necesita mover los embeddings a un servicio dedicado, el patrón Repository ya tiene aislada esa lógica.

13. Tecnologías, lenguaje y dependencias

13.1 Lenguaje principal

Todo el proyecto está escrito en JavaScript con módulos ES (ESM), sin TypeScript. Tanto el frontend como el backend declaran "type": "module" en su package.json, lo que permite usar import/export en todo el código. Sequelize CLI usa archivos .cjs únicamente donde la herramienta lo exige.

13.2 Frontend

Tecnología	Versión	Rol
React	^19.2.5	Biblioteca principal para construcción de UI
Vite	^8.0.10	Herramienta de build y servidor de desarrollo
React Router	^7.14.2	Navegación entre páginas (SPA)
Bootstrap + React Bootstrap	^2.10.10	Sistema de estilos base y componentes UI (modales, dropdowns)
Mapbox GL	^3.23.1	Mapa interactivo de tiendas
Recharts	^3.8.1	Gráficos de estadísticas del jugador
Lucide React	^1.14.0	Íconos de la interfaz
Supabase JS	^2.105.1	Cliente para autenticación

13.3 Backend

Tecnología	Versión	Rol
Node.js	≥18 (ESM)	Runtime del servidor
Express	^5.2.1	Framework HTTP
Sequelize	^6.37.8	ORM para PostgreSQL
pg / pg-hstore	^8.20.0	Driver nativo de PostgreSQL
sequelize-cli	^6.6.5	Migraciones de base de datos
@supabase/supabase-js	^2.105.1	Admin SDK para verificar tokens JWT
Zod	^4.3.6	Validación de schemas de request
Resend	^6.12.3	Envío de correos transaccionales
p-limit	^7.3.0	Control de concurrencia (script embeddings)
dotenv	^17.4.2	Carga de variables de entorno
nodemon	^3.1.14	Hot reload en desarrollo

13.4 Base de datos e infraestructura

Servicio	Rol
PostgreSQL (Supabase)	Base de datos principal con todas las entidades del sistema
pgvector	Extensión que habilita búsqueda por similitud vectorial (Asistente IA)
Supabase Auth	Autenticación de usuarios (registro, login, JWT)
Vercel	Hosting del frontend con build automático y CDN global
Render.com	Hosting del backend como Web Service Node.js
Nomic AI	Generación de embeddings vectoriales (proceso offline)
DeepSeek	Acceso a deepseek-V4-flash para explicaciones del Asistente IA
Mapbox	Mapas interactivos
Resend	Notificaciones por correo electrónico

13.5 Estándares de código

Linter y formateador: el frontend tiene ESLint y Prettier integrados entre sí, con scripts de npm listos para ejecutar (lint, format). El backend también tiene ambas herramientas instaladas como dependencias de desarrollo.

Convenciones de nombres (consistentes en ambos repositorios):

- PascalCase para modelos de Sequelize y componentes de React (Usuario, MTGCard, DeckBuilder)
- camelCase con prefijo use para hooks de React (useAuth, useConfirmDialog)
- dominio.tipo.js en minúsculas para archivos del backend (mazos.service.js, torneos.routes.js, auth.controller.js)
- Sufijo Context para contextos de React (AuthContext, ToastContext)
- kebab-case para utilidades del frontend
- Estructura de carpetas: organización por dominio/módulo en lugar de por tipo de archivo, tanto en frontend (src/modules/{dominio}/...) como en backend (src/modules/{dominio}/{tipo}.js).

13.6 Repositorios y flujo de trabajo

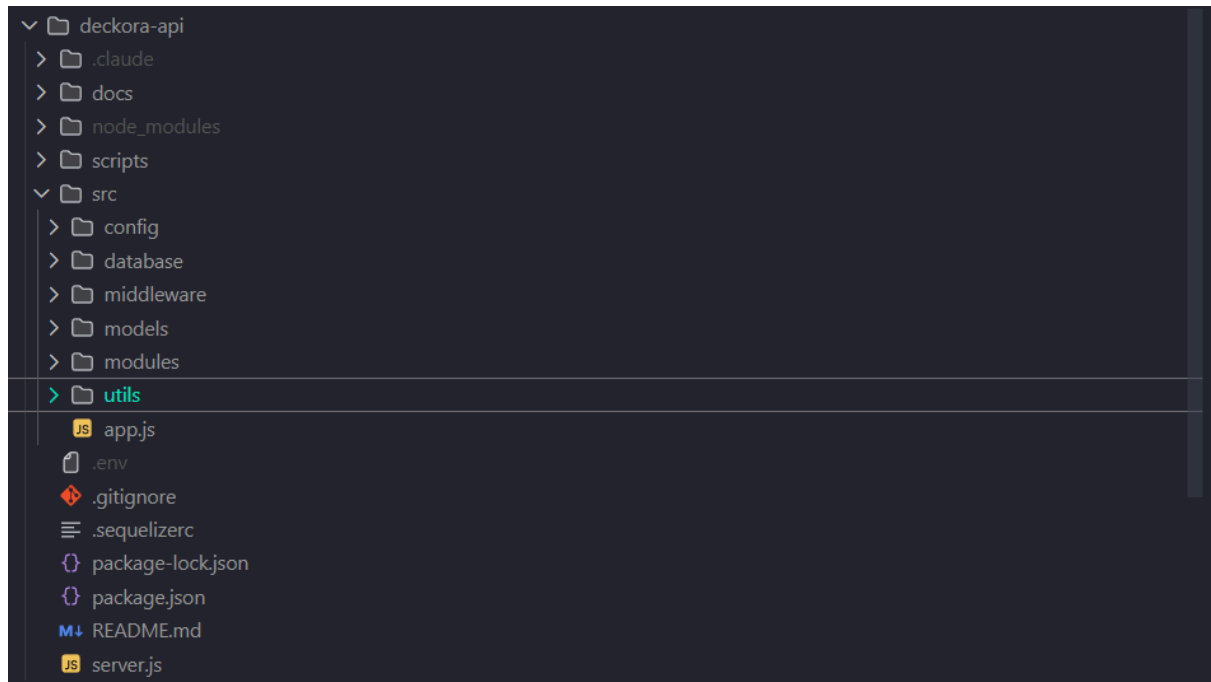
Repositorio	URL
Frontend (Deckora-Web)	https://github.com/VerdaMech/Deckora-Web
Backend (Deckora-API)	https://github.com/VerdaMech/Deckora-API

Flujo de ramas (Git Flow simplificado):

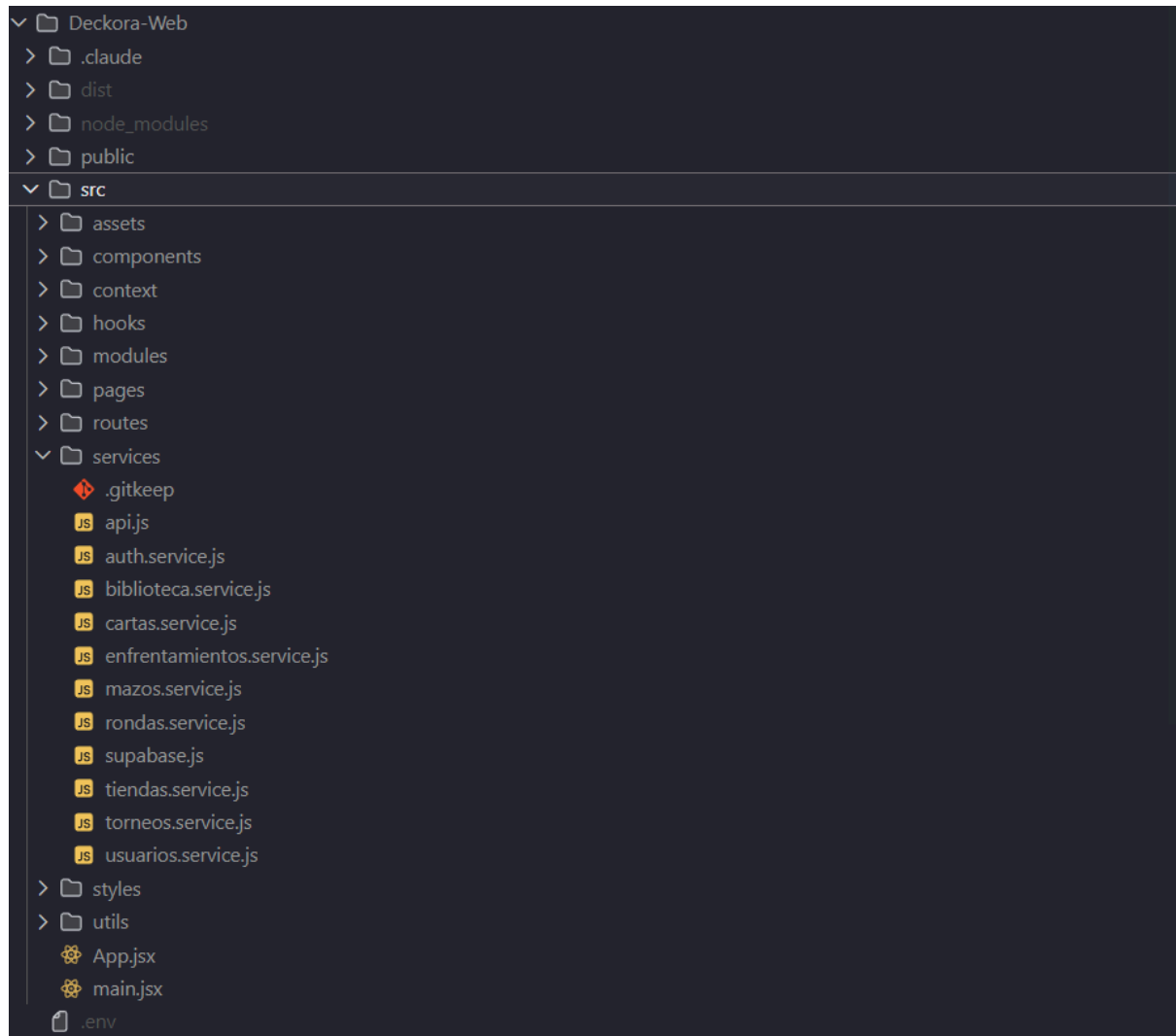
- main: rama de producción. Solo recibe merges desde dev mediante Pull Request revisada y aprobada.
- dev: rama de integración. Recibe merges desde ramas de feature o fix mediante Pull Request revisada y aprobada.
- feature/<nombre>: ramas creadas para agregar nuevas funcionalidades, partiendo de dev.
- fix/<nombre>: ramas creadas para corregir errores, partiendo de dev.

Política estricta: ningún cambio se hace push directo a dev ni a main. Toda integración pasa obligatoriamente por una Pull Request que el otro integrante debe revisar y aprobar antes del merge. Esto garantiza que cada cambio sea revisado al menos una vez antes de entrar a la rama de integración, y dos veces antes de llegar a producción.

Estructura de carpetas:

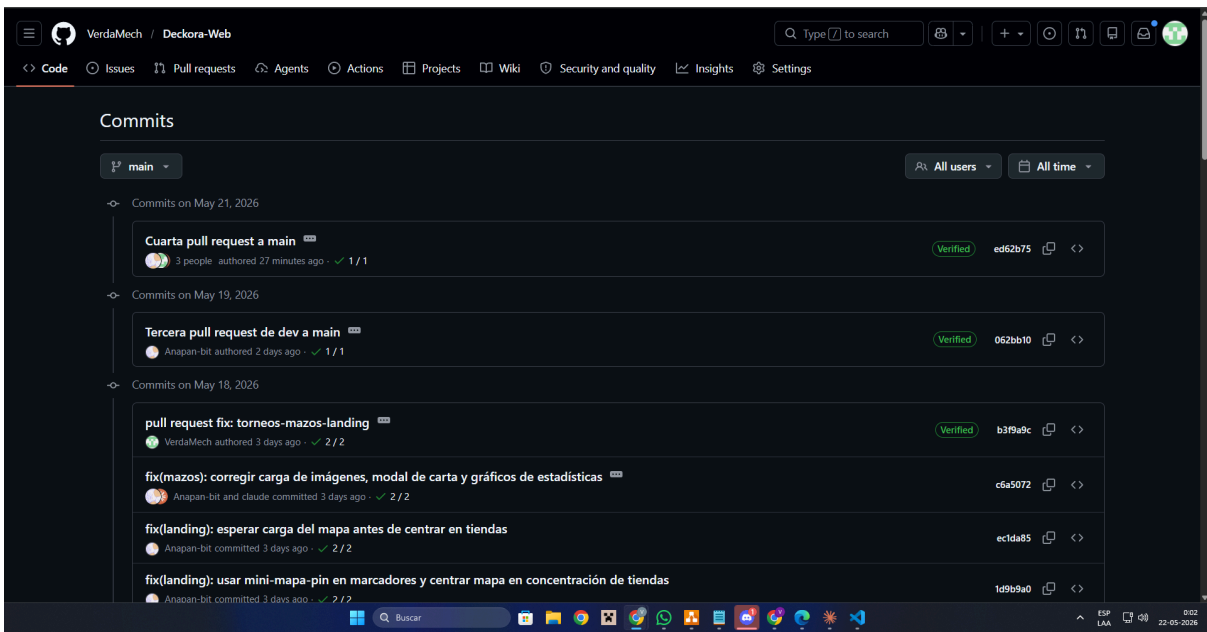


24. Estructura de carpetas: backend

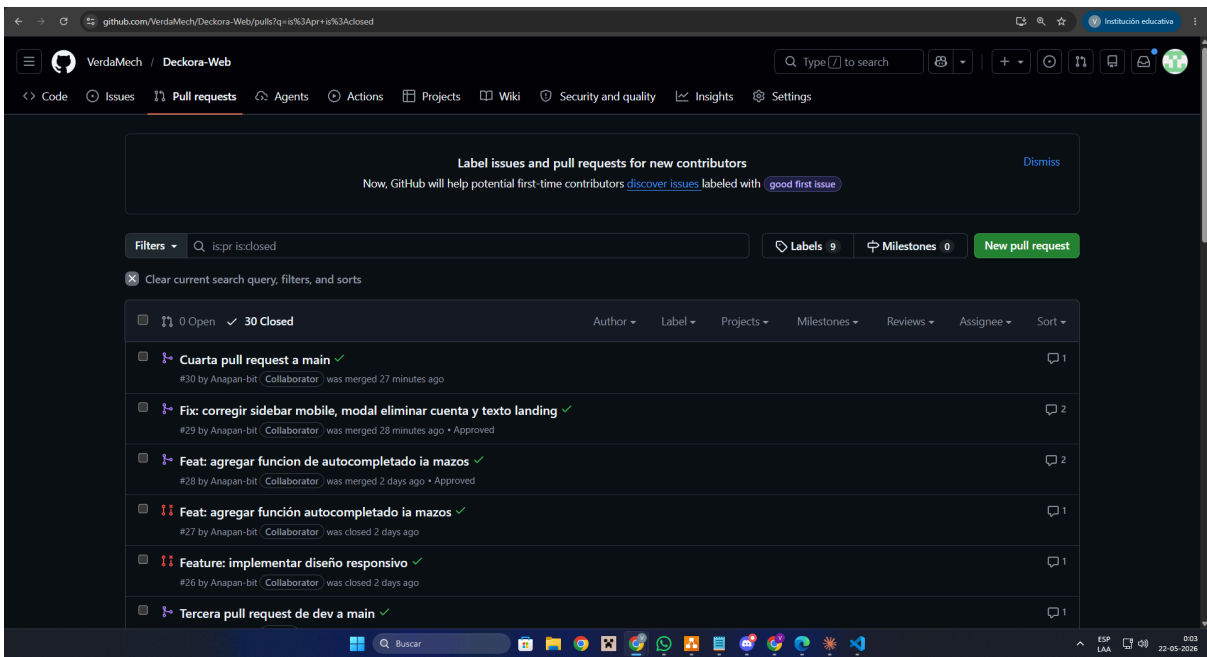


25. Estructura de carpetas: frontend

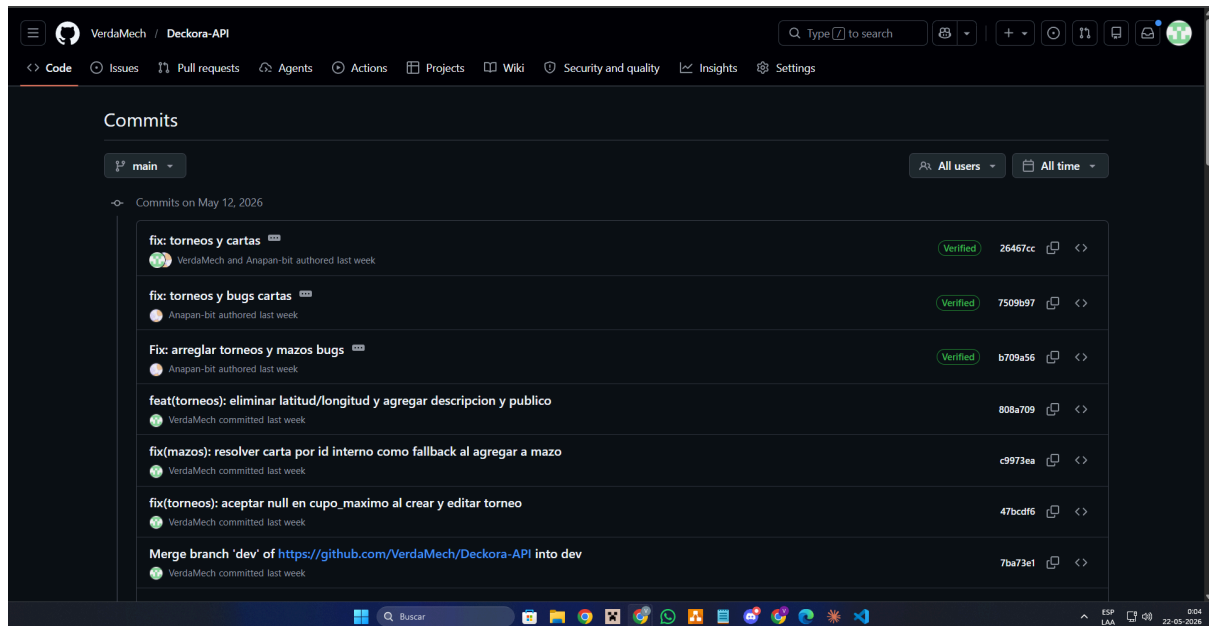
Evidencia de PRs y commits:



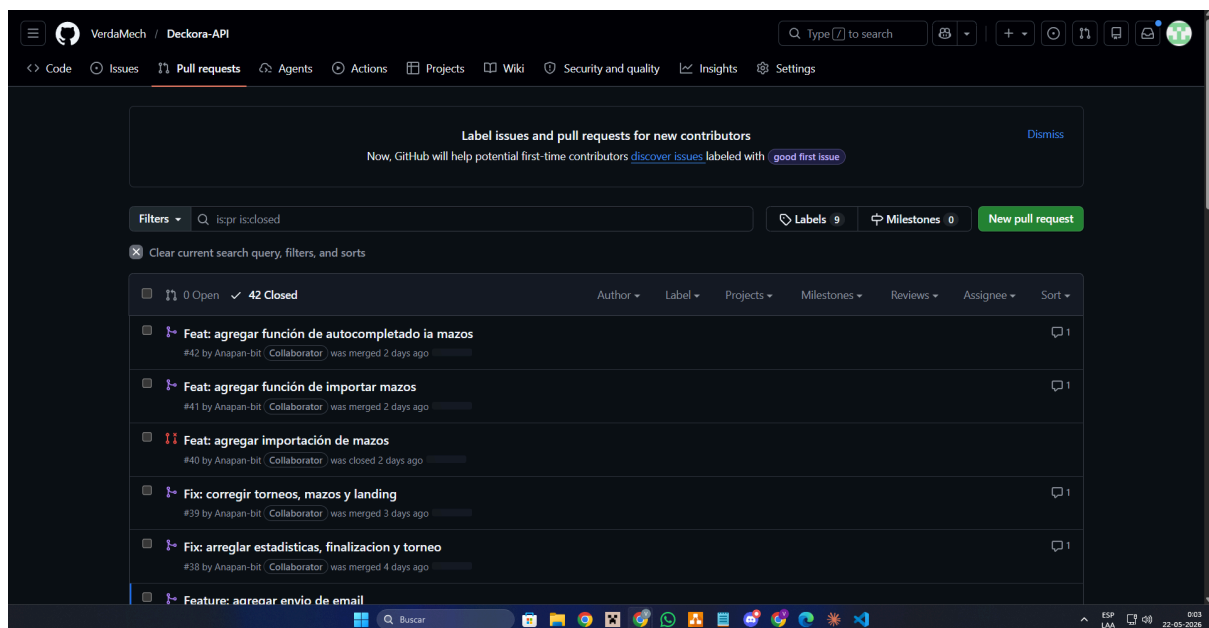
26. Historial commits: frontend



27. historial Pull requests: frontend



28. Historial commits: backend



29. Historial pull requests: backend

14. Configuración de servidor de producción (paso a paso)

14.1 Plataformas elegidas

Deckora se despliega en una arquitectura distribuida sobre servicios cloud, cada uno especializado en una capa del sistema:

Componente	Plataforma	Plan
Frontend (SPA)	Vercel	Hobby (gratis)
Backend (API)	Render.com	Free Web Service
Base de datos + Auth	Supabase	Free tier
Embeddings vectoriales	Nomic AI (Atlas)	Gratis con API key
LLM para explicaciones	DeepSeek	API key, pay as you go
Mapas	Mapbox	Free tier (50.000 cargas/mes)
Correo transaccional	Resend	Free tier (3.000 emails/mes)
Dominio personalizado	Registrador externo → Vercel	deckoratcg.cl

14.2 Requisitos técnicos del servidor

- **Runtime:** Node.js versión 18 o superior, con soporte de ES Modules.
- **Puerto:** asignado dinámicamente por Render vía variable de entorno PORT. Express lee process.env.PORT || 3001.

14.3 Variables de entorno backend (configuradas en Render):

- NODE_ENV=production
- DATABASE_URL (conexión a Supabase, PostgreSQL)
- SUPABASE_URL (URL del proyecto Supabase)
- SUPABASE_SERVICE_KEY (service_role secret key, nunca exponer al frontend)
- DEEPSEEK_API_KEY
- NOMIC_API_KEY
- RESEND_API_KEY

14.4 Variables de entorno frontend (configuradas en Vercel):

- VITE_SUPABASE_URL
- VITE_SUPABASE_ANON_KEY (pública)
- VITE_API_URL (URL del backend con sufijo /api)
- VITE_MAPBOX_TOKEN (token público de Mapbox)

Permisos de base de datos: el rol de la connection string debe tener permisos de lectura, escritura y ejecución de DDL (para migraciones).

14.5 Instrucciones paso a paso

Paso 1 — Configurar Supabase (base de datos y autenticación)

1. Crear un nuevo proyecto en supabase.com.
2. Activar la extensión pgvector (requerida para los embeddings del Asistente IA). En Dashboard → SQL Editor, ejecutar: CREATE EXTENSION IF NOT EXISTS vector;
3. Anotar las credenciales desde Settings → API: Project URL (será SUPABASE_URL y VITE_SUPABASE_URL), anon/public key (será VITE_SUPABASE_ANON_KEY), service_role secret key (será SUPABASE_SERVICE_KEY, solo backend), Connection string desde Settings → Database (será DATABASE_URL).

4. En Authentication → URL Configuration, agregar el dominio de la aplicación (<https://www.deckoratcg.cl> y <https://deckora-web.vercel.app>) a la lista de Redirect URLs para que funcione la recuperación de contraseña.

Paso 2 — Desplegar el backend en Render.com

1. Crear un nuevo Web Service en render.com.
2. Conectar el repositorio Deckora-API y seleccionar la rama main.
3. Configurar el servicio: Environment Node, Build command npm install, Start command npm start, Node version 18 o superior.
4. Agregar todas las variables de entorno del backend listadas arriba en Environment.
5. Una vez activo, ejecutar las migraciones desde la Shell del servicio (o localmente apuntando DATABASE_URL a Supabase): npm run db:migrate
6. Poblar la base de datos con cartas MTG (solo la primera vez): node scripts/seedCartas.js (importa ~32.500 cartas desde Scryfall) y luego npm run embed:generate (genera embeddings vectoriales, 10-20 min).

Paso 3 — Desplegar el frontend en Vercel

1. Importar el repositorio Deckora-Web en vercel.com.
2. Vercel detecta automáticamente que es un proyecto Vite. Verificar: Build command npm run build, Output directory dist, Install command npm install.
3. Agregar las variables de entorno frontend listadas arriba en Settings → Environment Variables.
4. El archivo vercel.json del repositorio ya incluye la regla de rewrite necesaria para que las rutas de SPA funcionen al recargar la página: { "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
5. Hacer el primer deploy. Cada push a main genera un nuevo deploy automáticamente.

Paso 4 — Configurar el dominio personalizado (opcional)

1. En Vercel → Project → Settings → Domains, agregar [deckoratcg.cl](https://www.deckoratcg.cl) y www.deckoratcg.cl.
2. En el panel del registrador de dominio, agregar los registros DNS que Vercel indica (CNAME o A records).
3. Vercel emite el certificado SSL automáticamente.

14.6 Logs y monitoreo básico

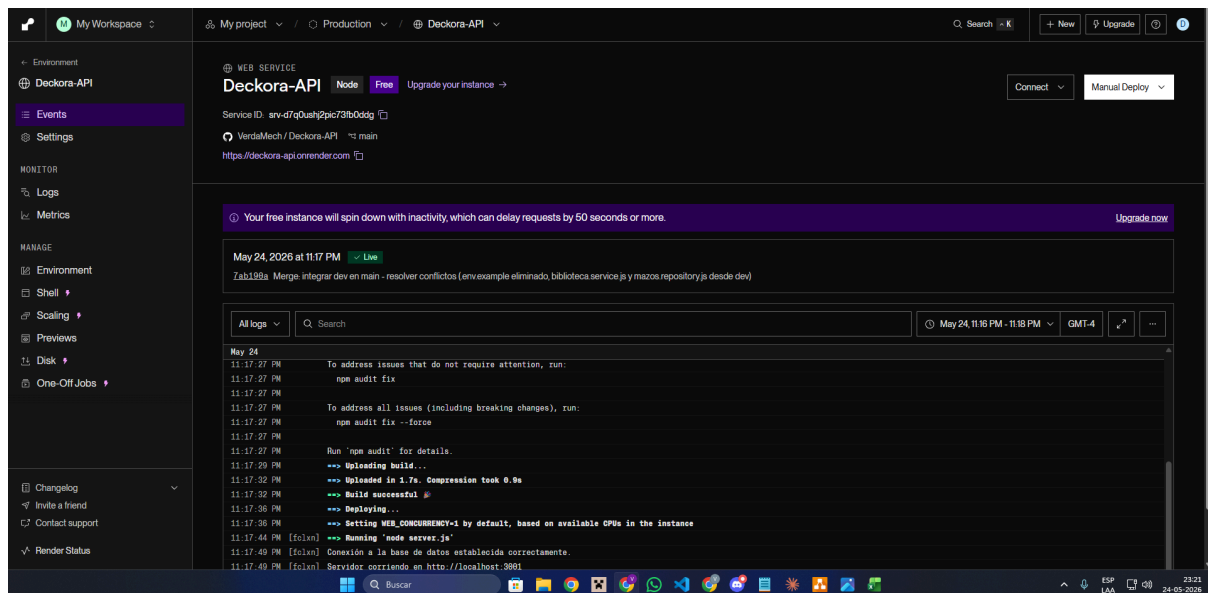
- **Logs del backend:** disponibles en Render → Service → Logs (en tiempo real).
- **Logs del frontend:** Vercel → Project → Deployments → cada deploy tiene su log de build y runtime.
- **Logs de base de datos:** Supabase → Logs → consultas SQL, autenticación, errores.

Nota importante: las variables `VITE_*` se incrustan en el bundle de Vite en tiempo de build. Cualquier cambio en ellas requiere disparar un nuevo deploy en Vercel para que surtan efecto.

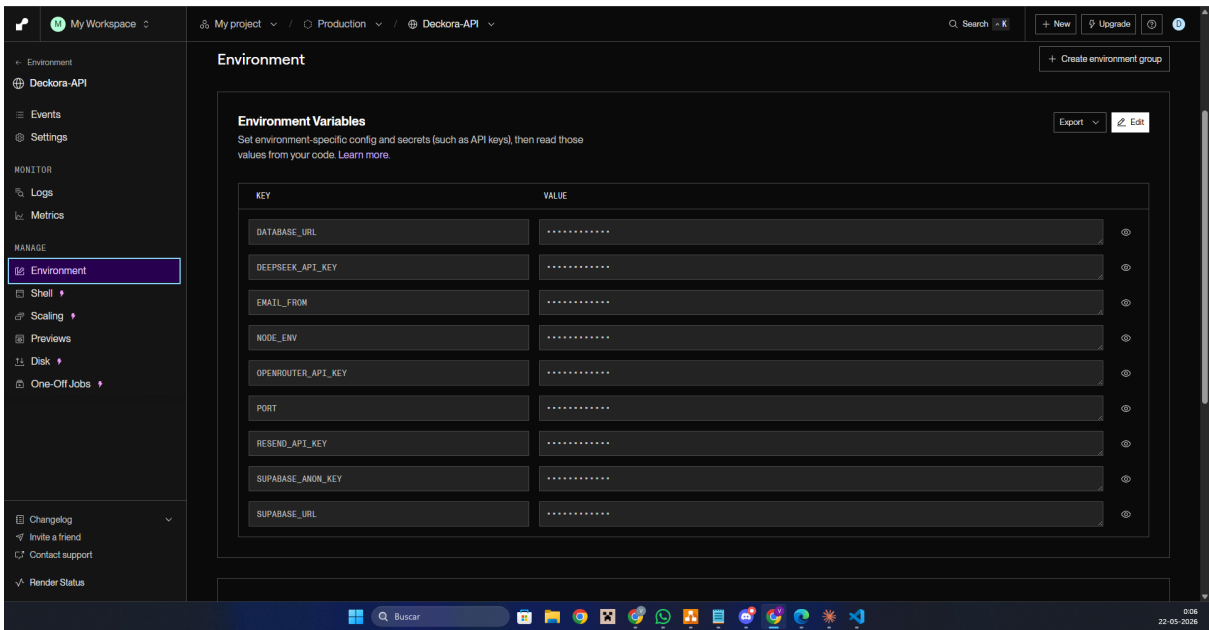
URLs de despliegue:

- **Frontend (dominio personalizado):** <https://www.deckoratcg.cl>
- **Frontend (dominio Vercel):** <https://deckora-web.vercel.app>
- **Backend API:** <https://deckora-api.onrender.com>

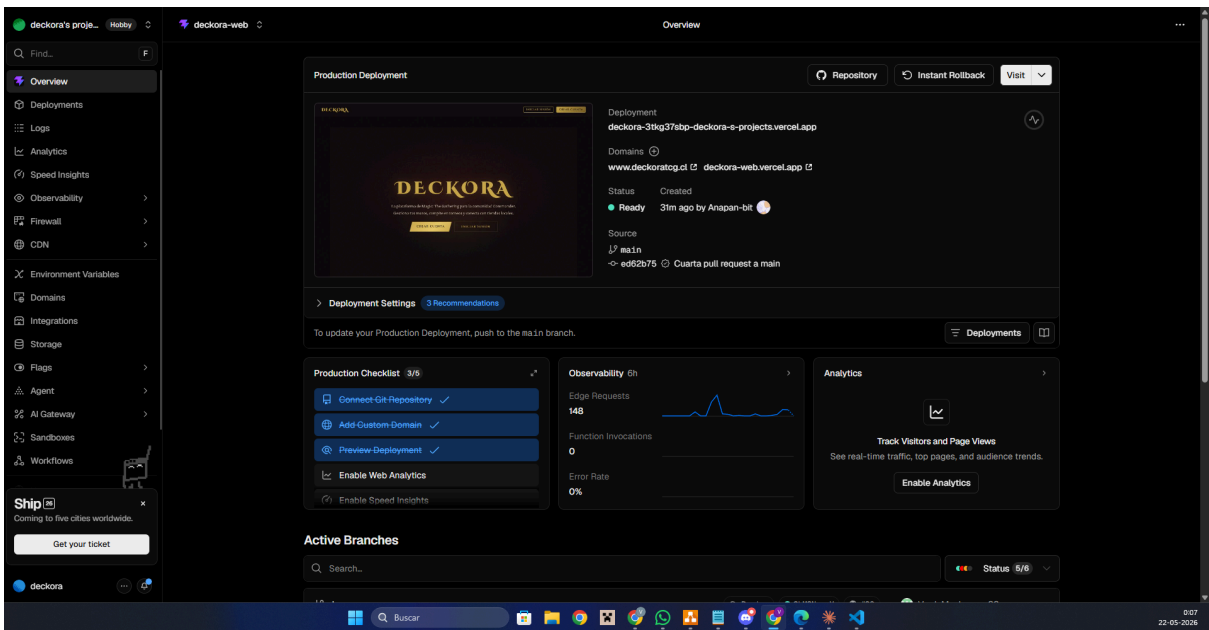
Capturas de configuración:



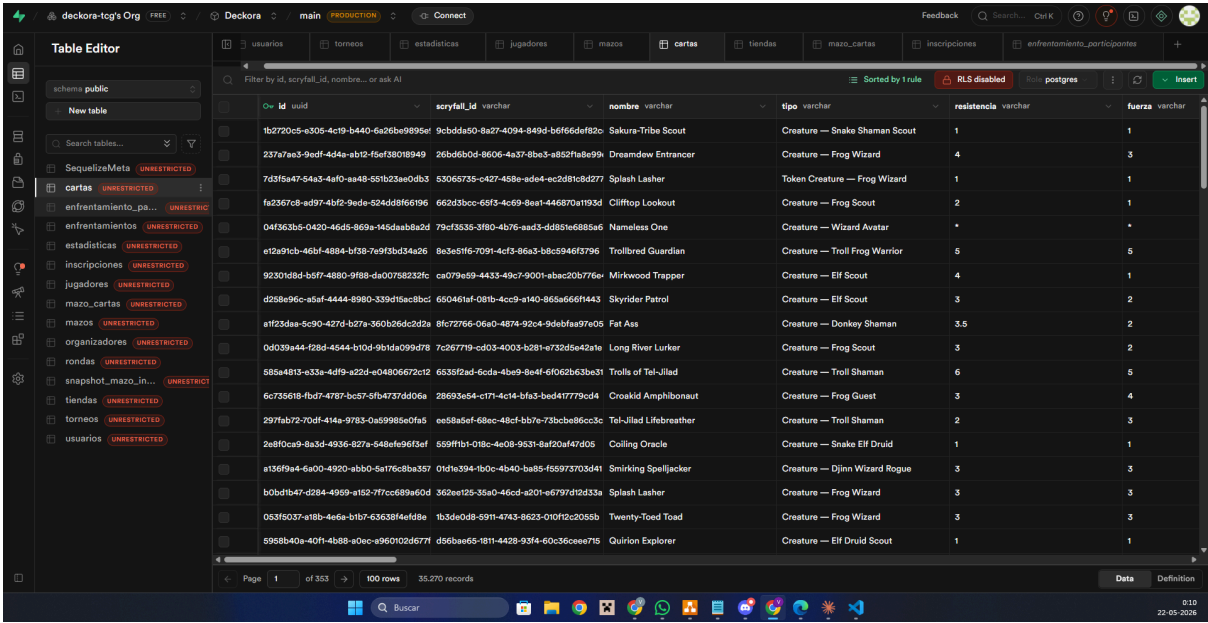
30. Render: registro de actividad



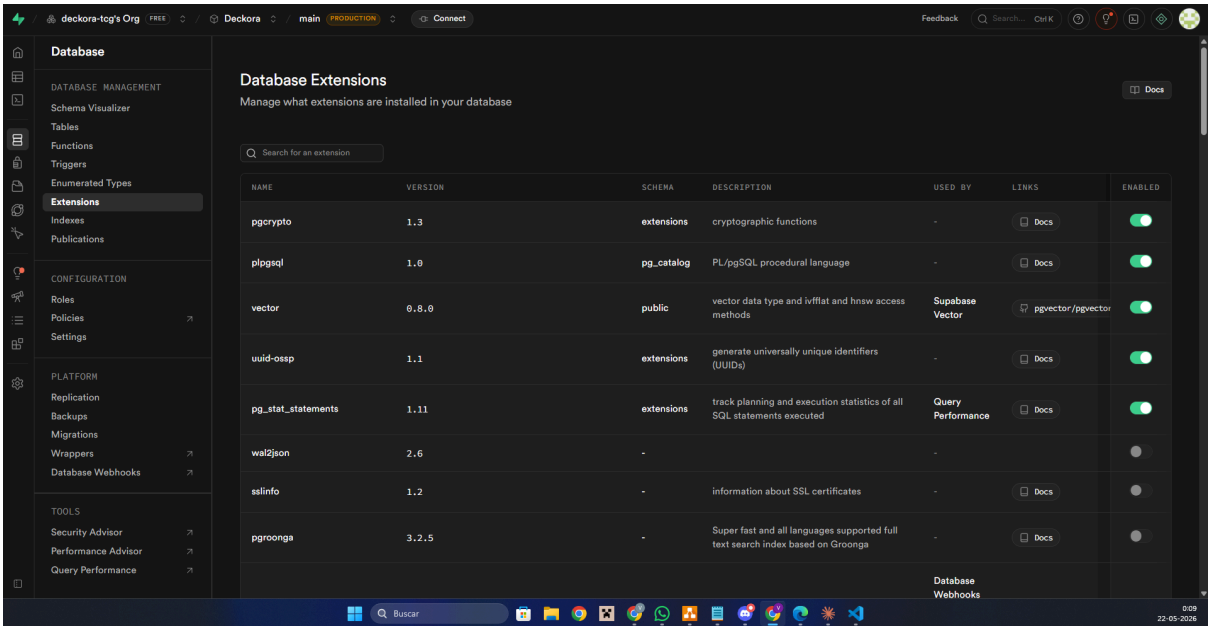
31. Render: variables de entorno



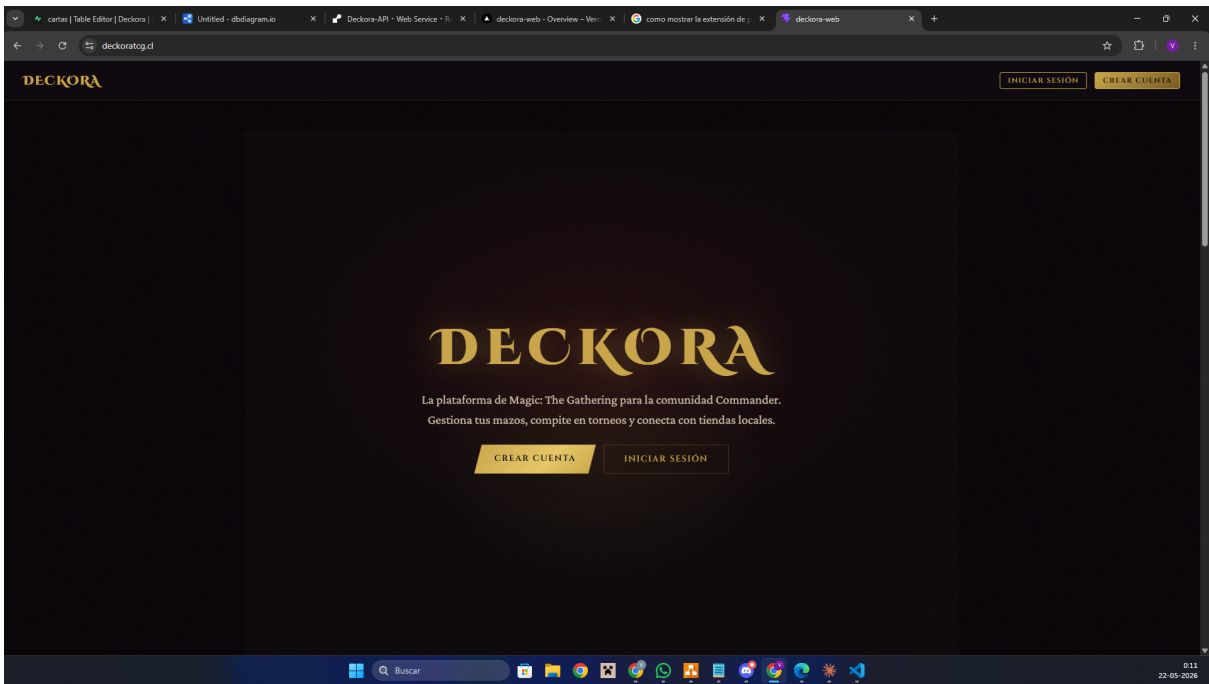
32. Vercel: panel de despliegue



33. Supabase: editor de tablas



34. Supabase: Extensiones utilizadas



35. Deckora desplegado

15. Estado de avance del desarrollo

Módulo de Identidad

Funcionalidad	Estado	Evidencia
Registro de usuario con rol (jugador/organizador/tienda)	[☒]	[]
Inicio de sesión con email + contraseña	[☒]	[]
Renovación automática de token JWT expirado	[☒]	[]
Recuperación de contraseña por correo	[☒]	[]
Perfil público de usuario en /u/:username	[☒]	[]

Módulo de Mazos

Funcionalidad	Estado	Evidencia
Crear mazo (nombre + formato)	[]	[]
Listar mazos del jugador	[]	[]
Agregar y quitar cartas de un mazo	[]	[]
Validación según formato (Commander, Standard, Modern, Pioneer, Legacy)	[]	[]
Asistente IA — recomendaciones vectoriales	[]	[]
Asistente IA — explicaciones en lenguaje natural	[]	[]
Actualización optimista al editar mazo	[]	[]

Módulo de Torneos

Funcionalidad	Estado	Evidencia
Crear torneo (organizador o tienda)	[]	[]
Editar torneo	[]	[]
Cartelera pública de torneos	[]	[]
Detalle de torneo con vista según rol	[]	[]
Inscripción de jugadores a torneos	[]	[]
Aprobar / rechazar inscripciones	[]	[]
Notificaciones por correo (inscripción, aprobación, rechazo)	[]	[]
Cambio de estado del torneo	[]	[]

Módulo de Rondas y Enfrentamientos

Funcionalidad	Estado	Evidencia
Crear rondas (Swiss, eliminación directa, final)	[]	[]
Emparejamiento automático Swiss	[]	[]
Emparejamiento manual (eliminación / final)	[]	[]
Reportar resultados de enfrentamientos	[]	[]
Tabla de posiciones por torneo	[]	[]

Módulo de Biblioteca y Cartas

Funcionalidad	Estado	Evidencia
Biblioteca pública de ~32.500 cartas MTG	[]	[]
Buscador de cartas en el editor de mazos	[]	[]

Módulo de Estadísticas

Funcionalidad	Estado	Evidencia
Estadísticas del jugador (mazos, torneos, victorias)	[]	[]
Gráficos con Recharts	[]	[]

Infraestructura y mapa

Funcionalidad	Estado	Evidencia
Mapa de tiendas en landing pública (Mapbox)	[]	[]
Selector de ubicación en perfil de tienda	[]	[]

Despliegue en Vercel + Render funcionando	[]	[]
---	---	-----

15.1 Calidad y seguridad aplicadas

Independientemente del estado específico de cada funcionalidad, el proyecto aplica las siguientes prácticas de calidad y seguridad de forma transversal:

- **Validación de entrada:** todos los endpoints del backend validan los datos del request con schemas de Zod antes de procesar.
- **Manejo de errores centralizado:** middleware errorHandler convierte excepciones con err.status en respuestas HTTP con el código y mensaje correctos. Los errores inesperados se loguean sin filtrar información sensible al cliente.
- **Control de acceso por capas:** middleware auth verifica el JWT en cada endpoint protegido. Middleware requirePerfil restringe el acceso por rol (jugador, organizador, tienda). Reglas de negocio en los servicios validan ownership (un jugador solo puede editar sus propios mazos, un organizador solo puede gestionar sus propios torneos).
- **Renovación automática de sesión:** si el JWT expira durante una request, el frontend renueva la sesión silenciosamente y reintenta la operación.
- **Variables sensibles fuera del bundle:** las claves de Supabase Service Role, DeepSeek, Nomic y Resend solo viven en el backend. Solo las claves públicas (anon key, Mapbox token) se incrustan en el bundle del frontend.
- **Fallo graceful en servicios externos:** si DeepSeek no responde, las recomendaciones del Asistente IA se entregan igual sin la explicación, en lugar de fallar.

16. Integraciones necesarias

Deckora depende de cinco integraciones externas para funcionalidades que serían costosas o complejas de construir desde cero. Todas tienen capa gratuita suficiente para el alcance actual del proyecto.

16.1 Supabase Auth — Autenticación de usuarios

Qué hace: gestiona el registro, inicio de sesión, emisión de tokens JWT y recuperación de contraseña. El backend no gestiona sesiones propias; únicamente verifica los tokens emitidos por Supabase usando la `service_role` key.

Datos que entran: email, contraseña, datos de perfil al registrar (nombre de usuario, rol).

Datos que salen: token JWT con identidad del usuario, refresh token para renovación.

Flujo técnico:

- Frontend llama a `supabase.auth.signInWithPassword()` → Supabase devuelve un JWT.
- Frontend inyecta el JWT en cada request: `Authorization: Bearer <token>`.
- Backend verifica el token con `supabase.auth.getUser(token)` usando la `service_role` key.

Consideraciones de seguridad:

- La `SUPABASE_SERVICE_KEY` vive únicamente en el backend y nunca se expone al cliente.
- La `VITE_SUPABASE_ANON_KEY` es pública por diseño; las políticas de Row Level Security en Supabase controlan qué datos puede leer/escribir cada usuario.
- Los tokens JWT tienen vigencia de 1 hora y se renuevan automáticamente.

Credenciales:

- SUPABASE_URL
- SUPABASE_SERVICE_KEY (backend)
- VITE_SUPABASE_URL
- VITE_SUPABASE_ANON_KEY (frontend).

16.2 Nomic AI — Embeddings vectoriales

Qué hace: transforma el texto descriptivo de cada carta MTG ("Instant CMC:1 Colors:R") en un vector de 768 dimensiones, permitiendo comparar cartas matemáticamente y encontrar similitudes semánticas sin reglas manuales.

Datos que entran: texto descriptivo de la carta (tipo, costo de maná, colores).

Datos que salen: vector de 768 floats.

Modelo: nomic-embed-text-v1.5

Endpoint: POST <https://api-atlas.nomic.ai/v1/embedding/text>

Proceso: offline, se ejecuta una sola vez al sembrar la base de datos (~10-20 min para las 32.500 cartas).

Consideraciones de seguridad:

- La NOMIC_API_KEY vive solo en backend, jamás se expone al cliente.
- Rate limits manejados con backoff exponencial (2s → 4s → 8s → 16s).
- Control de concurrencia con p-limit para no saturar la API.

Credencial: NOMIC_API_KEY (backend).

16.3 DeepSeek (Deepseek-V4-flash) — Explicaciones en lenguaje natural

Qué hace: Cumple con dos funciones distintas:

- Genera texto explicativo que acompaña a cada recomendación.
- Genera listas de cartas al momento que el usuario busca autocompletar un mazo existente.

Datos que entran: formato y cartas presentes en el mazo (nombre, tipo y texto). En la función de autocompletar, cantidad de cartas faltantes y comandante (si aplica)

Datos que salen: texto explicativo en español, o lista de cartas en la función de autocompletar.

Modelo: deepseek-V4-flash

Endpoint: POST <https://api.deepseek.com/chat/completions>

Modo de fallo: Si DeepSeek no responde o falla, las recomendaciones se entregan igualmente sin la explicación.

Consideraciones de seguridad:

- La DEEPSEEK_API_KEY vive solo en backend.
- Se invoca en tiempo de request, no se almacenan las respuestas del LLM (cada llamada es independiente).

Credencial: DEEPSEEK_API_KEY (backend).

16.4 Mapbox — Mapas interactivos

Qué hace: renderiza el mapa de tiendas en la landing pública y el selector de ubicación en la configuración de tiendas, permitiendo seleccionar coordenadas visualmente en lugar de ingresarlas a mano.

Datos que entran: coordenadas (lat, lng), marcadores de tiendas.

Datos que salen: mapa interactivo renderizado con marcadores.

Consideraciones de seguridad: El token de Mapbox es público por diseño, queda incrustado en el bundle del frontend. Esto es práctica estándar para tokens de Mapbox con permisos de lectura.

Capa gratuita: 50.000 cargas/mes (más que suficiente para el volumen actual).

Credencial: VITE_MAPBOX_TOKEN (frontend).

16.5 Resend — Correos transaccionales

Qué hace: envía notificaciones automáticas por correo electrónico ante tres eventos del flujo de inscripciones.

- Un jugador solicita inscribirse a un torneo (avisa al organizador).
- El organizador aprueba una inscripción (avisa al jugador).
- El organizador rechaza una inscripción (avisa al jugador).

Datos que entran: dirección de email del destinatario, plantilla con datos del torneo.

Datos que salen: correo HTML al buzón del destinatario.

Consideraciones de seguridad: La RESEND_API_KEY vive solo en backend.

Capa gratuita: 3.000 emails/mes. En desarrollo se puede usar el dominio @resend.dev sin verificación; en producción se debe verificar el dominio propio.

Credencial: RESEND_API_KEY (backend).

17. Estrategia y enfoque de calidad

Deckora gestiona datos competitivos clave: inscripciones, validación de mazos, emparejamientos, registro de resultados y tabla de posiciones. Un error en estas operaciones compromete los resultados de manera severa. Por eso, para este proyecto la calidad se define como **la corrección y la integridad de la lógica de negocio crítica**, sostenida por una infraestructura de pruebas mantenible y por el **aislamiento absoluto de la base de datos de producción**.

No se busca "probar todo", sino **probar las funciones cuyo funcionamiento es crítico**, priorizando por riesgo. Una cobertura del 90% bien enfocada en lógica crítica vale más que un 100 % sobre getters triviales.

17.2 Atributos de calidad evaluados (ISO/IEC 25010)

No todos pesan igual. La priorización refleja el dominio del proyecto:

Atributo	Prioridad	Cómo se aborda
Funcionalidad / Corrección	Alta	reglas de formato de mazos, emparejadores, ciclo de vida del torneo, registro de resultados
Fiabilidad	Alta	Atomicidad transaccional, idempotencia del registro de resultados, rollback ante errores
Seguridad	Alta	Autenticación (JWT Supabase), autorización por rol, validación de entrada (Zod), auditoría de dependencias
Rendimiento	Media	Pruebas de carga (k6) sobre endpoints de alta concurrencia
Mantenibilidad	Media	Cobertura medida, suite de regresión, CI como puerta de calidad, análisis estático
Usabilidad	Baja	Verificada indirectamente vía E2E (flujos completos de usuario)

17.3 Niveles y tipos de prueba (orden de aplicación)

Se aplicó la **pirámide de testing**, de lo más pequeño y barato a lo más completo y costoso:

Secuencia ejecutada:

1. **Unitarias:** Se probó lógica pura aislada, sin la BD o la red. Las 5 estrategias de validación de mazos (Commander, Standard, Modern, Pioneer, Legacy), los 3 emparejadores de rondas (Swiss, manual/bracket, aleatorio), los middlewares (auth, requirePerfil, validate) con Supabase mockeado, y las utilidades del frontend (validators, formatters, deck-helpers, torneos-helpers)
2. **Integración:** Se probó la lógica de negocio completa en la capa de servicios, mockeando sólo lo externo (BD, Supabase, email): auth.service (rollback ante correo duplicado), mazos.service, torneos.service (ciclo de vida e inscripciones) y enfrentamientos.service (puntos + estadísticas dentro de la misma transacción).
3. **API/contrato:** Se validó el contrato HTTP de cada endpoint: status codes, estructura JSON, validación Zod, autenticación (401) y autorización (403).
4. **Smoke:** Verificación mínima de que el sistema arranca (menos de 30 segundos) y sus endpoints vitales responden: GET /health, biblioteca y torneos sin auth, POST /api/mazos sin token, torneo inexistente.
5. **E2E:** Se recorrieron flujos completos en navegador real: registro, login válido/inválido, sección de mazos, creación de torneo, solicitud y aprobación de inscripción, transición de PENDIENTE a EN_CURSO para el estado de torneos, y bloqueo de un jugador en ruta de organizador.
6. **Regresión:** Se reejecutó la suite completa antes de cada merge. Los casos críticos (estrategias, middleware auth, services de torneos/enfrentamientos) se etiquetaron con @regression para correr de forma selectiva y evitar romper lo que ya funcionaba.
7. **Seguridad:** Auditoría de dependencias, análisis estático de patrones inseguros, y verificación de que toda ruta protegida devuelve 401/403. Se documentaron hallazgos y un checklist previo a producción.

8. **Carga:** Se midió p95/p99 y throughput en 4 escenarios (health, búsqueda de cartas, ciclo de torneo, inscripciones concurrentes). 2 fallaron por latencia de infraestructura free tier (no por el código: 0 % errores HTTP) y se detectó un bug real de race condition en inscripciones (500 en vez de 409).
9. **Integración en CI/CD — GitHub Actions:** Todo se integró con gates por rama. PR a dev ejecuta audit + lint + unit; merge a main corre la suite completa sobre un PostgreSQL efímero; y el push a main dispara un smoke contra la URL desplegada.

17.4 Enfoque

- **Mezcla mayoritariamente automatizada:** 1.312 pruebas automatizadas con Vitest (625 de backend y 687 de frontend), 10 specs E2E (Playwright) y 4 escenarios de carga (k6). La revisión de seguridad combina automatización (npm audit, ESLint Security) con un checklist manual previo a producción.
- **Datos de prueba:** fixtures en memoria con UUID ficticios y dominios inexistentes, **nunca** datos, tokens ni claves reales. Los servicios externos (Supabase Auth, Resend, DeepSeek, Nomic, Mapbox) se sustituyen por dobles de prueba (mocks/stubs/spies).
- **Ambientes:** unitarias e integración en local con mocks; API con la app Express en memoria; E2E contra local host (frontend + backend); carga contra una instancia de test o local con .env.test

Regla crítica: ningún test toca la base de datos de producción.

17.5 Definición de "Hecho" (Definition of Done)

Una funcionalidad se considera **realmente validada** cuando cumple **todas** estas condiciones:

- Tiene casos de prueba diseñados (en la matriz) y **todos pasan**.
- No tiene **defectos abiertos de severidad alta o crítica**.
- Cumple el **umbral de cobertura** de su fase (lógica crítica $\geq 70\%$ en ramas cubiertas).
- Pasa los **gates de CI** (audit + lint + suite correspondiente a la rama).
- Está **documentada** en el plan, el reporte y la matriz de trazabilidad.

18. Plan de pruebas

18.1 Alcance de las pruebas

Dentro de alcance: Niveles unitarios, integración, API/contrato, sistema (E2E), humo, seguridad y carga, sobre los dos componentes desplegables: **Deckora-api** y **Deckora-Web**.

Fuera de alcance:

Excluido	Justificación	Riesgo asumido
Recomendaciones IA (DeepSeek) y embeddings (Nomic/pgvector)	API externas de pago; costo por request	Bajo, funciones que no son críticas para el funcionamiento
Renderizado de mapa (Mapbox GL)	WebGL con token externo	Bajo: sólo visualización
Interno de Supabase Auth / Resend	Productos de terceros ya probados; se mockean Bajo: se valida la integración, no el producto	Bajo: se valida la integración, no el producto
Cross-browser y accesibilidad (a11y)	No es requisito de esta etapa; solo Chromium	Medio: se difiere a fase futura

18.2 Elementos y funcionalidades a probar

- **Nuevas:** validar mazo según formato (Commander, Standard, Modern, Pioneer, Legacy); inscribirse a un torneo; emparejar rondas (Swiss, eliminación, final); registrar resultados; avanzar el ciclo del torneo; aprobar/rechazar inscripciones; consultar tabla de posiciones.
- **Regresión:** autenticación y autorización, validación de formularios y todos los flujos P1 (mazos, inscripción, resultados), marcados con la etiqueta @regression para ejecutarse antes de cada merge.

18.3 Criterios de aceptación / rechazo

Tipo	Criterio de aceptación
Unitario	Resultado real = esperado; sin efectos secundarios; < 100 ms/test
Integración	Service correcto; cero llamadas reales a servicios externos
API	Status HTTP, cuerpo JSON y headers correctos
Smoke	Endpoints vitales responden en < 2 s; suite < 30 s
E2E	Flujo completo sin error y sin errores en consola
Carga	p95 < 500 ms · p99 < 1000 ms · tasa de error < 1 %
Seguridad	0 vulnerabilidades críticas/altas en npm audit
Global	Ejecutar el 100 % de las pruebas unitarias; suite de regresión 100 % verde en main

18.4 Criterios de suspensión y reanudación

- **Suspensión:** si > 30 % de los casos de una suite falla por una causa común (p. ej. configuración rota, BD de test caída), o si aparece un defecto que bloquea todo el flujo, se detiene la ejecución de esa suite y se prioriza corregir el entorno.
- **Reanudación:** cuando el smoke de la suite vuelve a pasar y la causa raíz está documentada.

18.5 Recursos, roles y cronograma (matriz RACI)

Actividad	Desarrollador	QA Lead	Docente guía
Diseño del plan	Consultado	Responsable/Aprobador	Informado
Implementación de tests	Responsable	Aprobador	Informado
Resolución de entornos (BD/auth de test)	Responsable	Aprobador	Consultado
Ejecución y reporte	Responsable	Aprobador	Informado
Aprobación de calidad	Consultado	Aprobador	Responsable

Cronograma: 10 fases en semanas

18.6 Premisas, dependencias y riesgos

- **Premisas:** el código compila y arranca; las dependencias de testing están instaladas; existe un entorno de test aislado.
- **Dependencias:** disponibilidad de una BD de test (D1) y de usuarios de prueba en un proyecto Supabase de test (D2) antes de las fases de integración y E2E.

ID	Riesgo	Nivel	Mitigación/Conatigencia
R-01	BD única dev/prod	Alto	BD de test aislada + salvaguarda por NODE_ENV; mocks de repositorio
R-02	Sin CI/CD: se despliega sin pruebas	Alto	Gates por rama en GitHub Actions (Fase 10), graduales
R-07	POST /api/torneos sin restricción de rol	Bajo	Documentado como hallazgo; decisión de producto pendiente
R-09	Sin rate limiting en auth	Bajo	express-rate-limit + prueba de fuerza bruta

19. Tipos de pruebas aplicadas

19.1 Pruebas unitarias y de componentes

Herramienta utilizada: Vitest

Tests realizados: 311 de 311 aprobados (backend) + 687 de 687 aprobados (frontend: 213 unitarias, 470 de componentes y 4 de páginas)

Propósito: verificar funciones y módulos de forma aislada, sin BD ni red.

Qué se probó: en el backend, las estrategias de validación de mazos (los 5 formatos, con sus reglas verificadas contra el código), los emparejadores (Swiss, manual/bracket, aleatorio, incluido el caso degenerado de 5 jugadores Commander), los middlewares (auth, requirePerfil, validate), los repositorios (con modelos Sequelize mockeados), los esquemas de validación Zod, los servicios de apoyo (email/plantillas) y utilidades (haversine, scryfallMapper); y en el frontend, sus utilidades (validators, formatters, deck-helpers, torneos-helpers) junto con sus componentes y páginas React, probados con Testing Library + jsdom.

Por qué es pertinente: es lógica pura crítica y de máximo ROI — bloquea inscripciones inválidas y errores de emparejamiento antes de que toquen datos reales.

Cobertura alcanzada: medida con `@vitest/coverage-v8` sobre todo `src/`. Backend: **91,1 %** sentencias, **82,2 %** ramas, **93,1 %** funciones, **91,6 %** líneas. Frontend: **90,2 %** sentencias, **78,3 %** ramas, **84,9 %** funciones, **92,3 %** líneas.

19.2 Pruebas de humo (smoke)

Herramienta utilizada: Supertest

Tests realizados: 5 de 5 tests aprobados

Propósito: confirmar en menos de 30 segundos que el sistema arrancó y sus endpoints vitales responden, antes de seguir probando.

Qué se probó: GET `/health` → 200; GET `/api/biblioteca/cartas` y GET `/api/torneos` → 200 sin auth; POST `/api/mazos` sin token → 401; GET `/api/torneos/:id` inexistente → 404.

Por qué: sirve de gate de despliegue rápido y como verificación post-deploy contra la URL de Render.

19.3 Pruebas de integración

Herramienta utilizada: Vitest (mocks de repositorio)

Tests realizados: 163 de 163 tests aprobados

Propósito: validar la lógica de negocio completa en la capa de servicios, mockeando sólo lo externo (BD, Supabase, email). Qué se probó: auth.service (signup con rollback ante correo duplicado), mazos.service (propiedad del mazo → 403), torneos.service (ciclo de vida e inscripciones: mínimo de jugadores, formato del mazo, no finalizar con enfrentamientos pendientes), y enfrentamientos.service (empate = 1 punto c/u; estadísticas actualizadas dentro de la misma transacción).

Por qué: estos flujos combinan varias capas y transacciones multi-tabla; es donde un error produce datos inconsistentes.

19.4 Pruebas funcionales / API (contrato HTTP)

Herramienta utilizada: Supertest

Tests realizados: 146 de 146 tests aprobados

Propósito: validar el contrato de cada endpoint: status code, estructura JSON, validación Zod, autenticación y autorización. Qué se probó: rutas protegidas sin token → 401; rol incorrecto → 403; body inválido → 400 con { error, detalles }; rutas públicas → 200; creación con body válido → 201. Aquí se verificaron correcciones respecto al plan v1.0 (p. ej. el resultado se registra con PATCH, no POST; crear torneo no exige rol — hallazgo R-07).

Por qué: define el contrato cliente-servidor que consume Deckora-Web.

19.5 Pruebas de extremo a extremo (E2E)

Herramienta utilizada: Playwright + Chromium

Tests realizados: 10 specs

Propósito: recorrer flujos completos en un navegador real, como un usuario.

Qué se probó: carga sin errores de consola, registro, login válido/inválido, sección de mazos, creación de torneo, solicitud y aprobación de inscripción, transición PENDIENTE → EN_CURSO, y bloqueo de acceso de un jugador a una ruta de organizador (→ /forbidden). **Por qué:** es la verificación más cercana al uso real y detecta problemas de integración Web + API + Auth.

19.6 Pruebas de regresión

Herramienta utilizada: Playwright + Vitest

Tests realizados: 1.322 acumuladas (1.312 pruebas Vitest + 10 specs E2E)

Propósito: asegurar que lo que ya funcionaba sigue funcionando tras cada cambio.

Qué se probó: los casos críticos marcados con @regression (estrategias Commander/Standard, Swiss, middleware auth, services de torneos y enfrentamientos) se ejecutan completos antes de cada merge.

Por qué: previene romper la lógica crítica al corregir otra cosa.

19.7 Pruebas de rendimiento / carga

Herramienta utilizada: K6

Tests realizados: 2 de 4 aprobados

Propósito: medir el comportamiento bajo demanda (p95/p99, throughput, tasa de error). **Qué se probó:** health check (10 VU), búsqueda de cartas (50 VU), ciclo de torneo (hasta 200 VU) e inscripciones concurrentes (hasta 40 VU).

Resultado: 2 escenarios pasaron y **2 fallaron**, principalmente por la infraestructura free tier de Render/Supabase (no por el código, que mantuvo 0 % de errores HTTP). La carga reveló además un bug real de race condition en inscripciones.

Por qué: aunque el público objetivo es reducido (torneos ≤ 40 jugadores), validar la capacidad de escala documenta los límites del entorno actual.

19.8 Pruebas de seguridad

Herramienta utilizada: npm audit + ESLint Security + Supertest

Propósito: Revisar autenticación, control de acceso, validación de entrada y manejo de secretos. **Qué se probó:** auditoría de dependencias (npm audit --audit-level=moderate, sin vulnerabilidades críticas/altas); análisis estático con eslint-plugin-security; y, vía tests de API, que toda ruta protegida devuelve 401/403 sin credenciales válidas. Se documentaron hallazgos (CORS sin whitelist, falta de rate limiting, POST /api/torneos sin rol) y un checklist previo a producción. **Por qué:** la seguridad protege toda la plataforma y los datos de los usuarios.

20. Herramientas y software utilizados

Cada herramienta fue seleccionada y justificada frente a alternativas (no se probó "a ojo"). La decisión más relevante: el backend declara "type": "module" (ESM nativo), lo que descartó Jest (soporte ESM experimental) y motivó Vitest.

Herramienta	Tipo de prueba / uso	Justificación de la elección
Vitest	Unitarias e integración (front y back)	Soporte ESM nativo (el backend es ESM puro); API compatible con Jest; comparte configuración con Vite en el frontend. Se eligió sobre Jest por el ESM.
@vitest/coverage-v8	Reportes de cobertura	Reportes HTML/JSON/LCOV integrados con Vitest;
Supertest API / contrato HTTP y smoke	Api/contrato HTTP y smoke	Levanta la app Express en memoria (sin desplegar); compatible con Express 5 y ESM.
@testing-library/react + jsdom + jest-dom	Componentes React	Estándar de la comunidad React; prueba comportamiento.
Playwright (@playwright/test) E2E	E2E	Multi-browser, codegen, trazas/videos/screenshots automáticos en fallo, robusto en CI. Se eligió sobre Selenium/Cypress por trazas y soporte ESM.
k6	Carga y rendimiento	Open source, scripts en JS, métricas p95/p99 y thresholds integrados; independiente de Node. Se eligió sobre JMeter por scripting en JS.

npm audit	Seguridad (dependencias)	Dependencias (incluido en npm) Integrado, cero configuración; gate directo en CI.
eslint-plugin-security	Seguridad (análisis estadístico)	Detecta patrones inseguros (regex peligrosas, eval, inyección) en código propio.
GitHub Actions	CI/CD gates por rama	Integrado al repositorio; gates de audit + lint + tests por PR; service container PostgreSQL efímero para integración.
GitHub (repos + ramas)	Gestión de casos/incidencias	Trazabilidad de bugs y ramas por fase (test/unit-tests-fase-2, etc.).

20.1 Herramienta de gestión documental de pruebas

La matriz de casos de pruebas (Revisar anexo de matriz de casos de prueba) centraliza 1.312 casos en 4 hojas (Backend, Frontend, Catálogo RF-RNF y Nota IDs), con columnas ID Original, ID Estandarizado, Repositorio, Módulo, Tipo de Prueba, Descripción, Archivo de Test, Grupo (Describe), Estado y RF/RNF.

21. Ejecución de pruebas y resultados

Tras ejecutar la batería completa de pruebas, el sistema presenta los resultados que se resumen a continuación. La tabla agrupa, por tipo de prueba, el número de casos, los exitosos, los fallidos y la cobertura o nota asociada; las cifras corresponden a la última ejecución de las suites automatizadas (Vitest, Supertest, Playwright y k6) sobre las ramas vigentes de Deckora-api y Deckora-Web.

Tipo de prueba	N.º de casos	Exitosos	Fallidos	Cobertura / Notas
Unitarias — backend (Vitest)	311	311	0	Estrategias, emparejadores, middlewares, repositorios, esquemas Zod, servicios y utilidades
Unitarias — frontend (Vitest)	213	213	0	Validators, formatters, deck/torneos-helpers y demás utilidades
Componentes y páginas — frontend (Vitest + Testing Library)	474	474	0	470 de componentes + 4 de páginas React
Integración — backend (Vitest)	163	163	0	Servicios con BD, Supabase y email mockeados
API / contrato (Supertest)	146	146	0	Status, JSON, Zod, 401/403 (17 enfocados en seguridad)
Humo / smoke (Supertest)	5	5	0	Arranque y endpoints vitales en < 30 s
E2E (Playwright +	10	10	0	Flujos completos en

Chromium)				navegador real
Carga / rendimiento (k6)	4	2	2	2 fallos por infraestructura free tier (0 % errores HTTP); reveló una race condition
TOTAL	1.326	1.324	2	99,8 % de éxito (los 2 fallos de carga son de infraestructura, no de código)

Indicadores globales: 1.324 de 1.326 casos exitosos (99,8 %). Los dos únicos fallos pertenecen a escenarios de carga limitados por la infraestructura de capa gratuita (Render/Supabase) y no a defectos del código, que mantuvo 0 % de errores HTTP. La cobertura de código medida con v8 alcanza, en el backend, 91,1 % de sentencias, 82,2 % de ramas, 93,1 % de funciones y 91,6 % de líneas; y en el frontend, 90,2 % de sentencias, 78,3 % de ramas, 84,9 % de funciones y 92,3 % de líneas, holgadamente por encima del umbral de la Definición de Hecho (≥ 70 % en lógica crítica). Toda la suite automatizada (1.312 pruebas Vitest + 10 specs E2E) se ejecuta en verde antes de cada merge a main.

Estos resultados se respaldan con evidencia concreta: capturas de los reportes de Vitest y del panel de cobertura HTML (v8) de backend y frontend, el reporte de Playwright, las métricas p95/p99 y de throughput de k6, y la salida de npm audit.

21.1 Evidencia de pruebas y resultados

Los números de la tabla anterior no son declarativos: cada cifra está respaldada por la salida real de la herramienta que la produjo. A continuación se presentan las evidencias capturadas en la última ejecución de las suites, organizadas por herramienta. En conjunto demuestran que la suite automatizada se ejecuta de forma reproducible, en verde y con la cobertura comprometida en la Definición de Hecho.

21.1.1 Ejecución de la suite del backend

```
Test Files 67 passed (67)
Tests      625 passed (625)
Start at   20:35:55
Duration   5.20s (transform 5.56s, setup 776ms, import 49.89s, tests 5.95s, environment 7ms)
```

36. Cantidad total de test backend

21.1.2 Panel de cobertura del backend

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	91.1	82.22	93.13	91.62	
src	100	100	100	100	
app.js	100	100	100	100	
src/config	100	50	100	100	
db.js	100	50	100	100	11
supabase.js	100	100	100	100	
src/middleware	100	100	100	100	
auth.js	100	100	100	100	
errorHandler.js	100	100	100	100	
requirePerfil.js	100	100	100	100	
validate.js	100	100	100	100	
src/models	100	100	100	100	
Carta.js	100	100	100	100	
Enfrentamiento.js	100	100	100	100	
EnfrentamientoParticipante.js	100	100	100	100	
Estadistica.js	100	100	100	100	
Inscripcion.js	100	100	100	100	
Jugador.js	100	100	100	100	
Mazo.js	100	100	100	100	
MazoCarta.js	100	100	100	100	
Organizador.js	100	100	100	100	
Ronda.js	100	100	100	100	
SnapshotMazoInscripcion.js	100	100	100	100	
Tienda.js	100	100	100	100	
Torneo.js	100	100	100	100	
Usuario.js	100	100	100	100	
index.js	100	100	100	100	
src/modules/auth	94.87	88.88	88.88	94.87	
auth.controller.js	86.2	87.5	80	86.2	5-9
auth.routes.js	100	100	100	100	
auth.schema.js	100	100	100	100	
auth.service.js	100	89.47	100	100	21,55
src/modules/biblioteca	96	79.31	100	97.95	
biblioteca.controller.js	88.88	100	100	88.88	19
biblioteca.routes.js	100	100	100	100	
biblioteca.schema.js	100	100	100	100	

backend* 0 0

37. Porcentaje de cobertura de test backend

21.1.3 Ejecución de la suite del frontend

```
Test Files 72 passed (72)
Tests      687 passed (687)
Start at   20:36:36
Duration   34.44s (transform 14.92s, setup 69.14s, import 136.71s, tests 42.57s, environment 228.71s)
```

38. Cantidad total de tests frontend

21.1.4 Panel de cobertura frontend

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	90.01	78.33	84.75	92.12	
components	84	73.68	100	90.47	
AuthCallbackHandler.jsx	84	73.68	100	90.47	10-11
components/domain	91.3	78.67	83.83	93.1	
CartaDetalleModal.jsx	100	81.39	100	100	14-15,24,27,34,60
CommanderBadge.jsx	100	100	100	100	
DeckBuilder.jsx	100	66.66	100	100	44-60
DeckList.jsx	93.75	77.14	84.61	93.1	92-103
DeckStats.jsx	100	70	100	100	49-55,64-65,106
EstadisticasJugador.jsx	86.04	72.34	73.33	88.09	26-31,69,139
EstadoBadge.jsx	100	100	100	100	
FormatBadge.jsx	100	100	100	100	
MTGCard.jsx	85.18	81.08	60	84.61	69,101-102,133
ManaCost.jsx	100	93.75	100	100	17
MapaTiendas.jsx	85.71	74.07	78.57	92	70-73
MiniMapaTienda.jsx	93.75	62.5	100	100	11-15
PodTable.jsx	81.81	72.72	62.5	84.21	42-43,102
RoleBadge.jsx	100	100	100	100	
RoundView.jsx	100	61.11	100	100	14-21,35
StorePin.jsx	100	100	100	100	
TournamentCard.jsx	100	100	100	100	
components/layout	80.55	93.18	68.18	80	
AppLayout.jsx	100	100	100	100	
Footer.jsx	100	100	100	100	
Navbar.jsx	70	95.23	50	70	92-102,126-127
Sidebar.jsx	91.66	87.5	87.5	90.9	23
components/ui	97.32	87.83	95	97.95	
Alert.jsx	100	100	100	100	
Badge.jsx	100	60	100	100	22-23,25,33-35
Button.jsx	100	100	100	100	
Card.jsx	100	100	100	100	
ConfirmDialog.jsx	100	80	100	100	24,32
EmptyState.jsx	100	100	100	100	
ErrorBoundary.jsx	100	100	100	100	
ErrorChunk.jsx	100	100	100	100	
Input.jsx	100	100	100	100	
Modal.jsx	100	100	100	100	
Select.jsx	100	88.88	100	100	52-53

39. Porcentaje de cobertura de test frontend

21.1.5 Reporte de playwright (E2E)

✓ smoke/smoke.spec.js		
✗ Smoke E2E	— carga de la aplicación › TC-E2E-001: la app carga en / sin errores de consola críticos	31.1s
smoke/smoke.spec.js:8 View Trace		
✓ auth/autenticacion.spec.js		
✓ TC-E2E-002: registro de jugador nuevo › muestra pantalla de verificación o redirige al dashboard tras registro válido		4.8s
auth/autenticacion.spec.js:12		
✓ TC-E2E-003: login con credenciales válidas › redirige a /jugador tras login exitoso		1.0s
auth/autenticacion.spec.js:46		
✓ TC-E2E-004: login con credenciales inválidas › muestra mensaje de error al ingresar contraseña incorrecta		989ms
auth/autenticacion.spec.js:103		
✓ mazos/mazos.spec.js		
✓ TC-E2E-005: jugador accede a mazos y puede crear uno › la sección de mazos carga correctamente para un jugador autenticado		3.9s
mazos/mazos.spec.js:10		

40. Resultados tests e2e parte 1

✓ torneos/torneos.spec.js		
✓ TC-E2E-006: organizador accede al formulario de creación de torneos › el formulario de creación carga y acepta datos básicos		3.9s
torneos/torneos.spec.js:20		
✓ TC-E2E-007: jugador solicita inscripción en torneo › el botón "Solicitar inscripción" aparece para jugador en torneo PENDIENTE		1.1s
torneos/torneos.spec.js:55		
✓ TC-E2E-008: organizador aprueba inscripción pendiente › el botón "Aprobar" aparece y la inscripción se aprueba		1.0s
torneos/torneos.spec.js:123		
✓ TC-E2E-009: organizador publica torneo cambiando estado a EN_CURSO › "Publicar torneo" aparece para organizador y completa la transición de estado		1.3s
torneos/torneos.spec.js:186		
✓ TC-E2E-010: jugador es bloqueado en ruta protegida de organizador › navegar a /organizador/torneos/nuevo redirige a /forbidden		762ms
torneos/torneos.spec.js:241		

41. Resultados tests e2e parte 2

21.1.6 Métricas de carga (k6)

Escenario	VUs	Duración	Requests	Errores	p95	p99	Estado
Health check (baseline)	10	30s	300	0.00%	1.04ms	–	✓ PASÓ
Búsqueda de cartas	50	2m 06s	1,694	0.00%	3.60s	4.19s	X UMBRAL
Ciclo de torneo	200	4m 12s	6,315	0.00%	11.26s	11.35s	X UMBRAL
Inscripciones concurrentes	40	2m 51s	207	0.00%	46.77s	49.77s	X UMBRAL
└ Exitosas (201)			34				
└ Ya inscrito (409)			93				
TOTAL	300		8,516	0.00%			

42. Resultados de tests de carga

21.1.7 Auditoría de dependencias

```
PS D:\Duoc\Tesis-Deckora\Tesis\deckora-api> npm run audit

> deckora-api@1.0.0 audit
> npm audit --audit-level=high

# npm audit report

uuid <11.1.1
Severity: moderate
uuid: Missing buffer bounds check in v3/v5/v6 when buf is provided - https://github.com/advisories/GHSA-w5hq-g745-h8pq
fix available via `npm audit fix --force`
Will install sequelize@3.30.0, which is a breaking change
node_modules/uuid
  sequelize 0.0.0-development || >=3.30.1
  Depends on vulnerable versions of uuid
  node_modules/sequelize

2 moderate severity vulnerabilities

To address all issues (including breaking changes), run:
npm audit fix --force
```

43. Resultados auditoría backend

La dependencia uuid (< 11.1.1) presenta una vulnerabilidad moderada heredada por sequelize. No se corrige porque el fix (npm audit fix --force) degradará Sequelize a la versión 3.30.0, un breaking change que comprometería la capa de datos del backend. El riesgo real es nulo ya que la vulnerabilidad solo es explotable al pasar buffers externos a uuid, algo que Sequelize no hace.

```
PS D:\Duoc\Tesis-Deckora\Tesis\Deckora-Web> npm audit
found 0 vulnerabilities
```

44. Resultados auditoría frontend

21.1.8 Análisis estadístico de seguridad

```
PS D:\Duoc\Tesis-Deckora\Tesis\deckora-api> npm run lint:security

> deckora-api@1.0.0 lint:security
> eslint src server.js && echo ✓ Análisis de seguridad completado: 0 vulnerabilidades encontradas

✓ Análisis de seguridad completado: 0 vulnerabilidades encontradas
```

45. Resultado de análisis de seguridad backend

```
PS D:\Duoc\Tesis-Deckora\Tesis\Deckora-Web> npm run lint:security

> deckora-web@0.0.0 lint:security
> eslint src --plugin security && echo ✓ Análisis de seguridad completado: 0 vulnerabilidades encontradas

D:\Duoc\Tesis-Deckora\Tesis\Deckora-Web\src\modules\identidad\pages\Registro.jsx
  46:10  warning  Potential timing attack, left side: true  security/detect-possible-timing-attacks

D:\Duoc\Tesis-Deckora\Tesis\Deckora-Web\src\modules\mazos\components\AsistenteIA.jsx
  36:10  warning  'resultadoAutocompletar' is assigned a value but never used  no-unused-vars

D:\Duoc\Tesis-Deckora\Tesis\Deckora-Web\src\modules\mazos\components\BarraAgregarCarta.jsx
  49:6   warning  React Hook useEffect has a missing dependency: 'formato'. Either include it or remove the dependency array  react-hooks/exhaustive-deps

X 3 problems (0 errors, 3 warnings)

✓ Análisis de seguridad completado: 0 vulnerabilidades encontradas
```

46. Resultado de análisis de seguridad frontend

Los tres warnings reportados por el análisis de seguridad del frontend no representan riesgos reales. El warning de detect-possible-timing-attacks en Registro.jsx es un falso positivo: la comparación password !== confirmacion se ejecuta del lado del cliente para validar un formulario, no para autenticar credenciales en el servidor, por lo que un timing attack no es aplicable. Los otros dos warnings son de reglas estándar de React, no de seguridad: una variable sin usar en AsistentelA.jsx y una dependencia faltante en un useEffect de BarraAgregarCarta.jsx, ambos sin impacto en la seguridad de la aplicación.

22. Conclusión

El análisis desarrollado a lo largo de este informe permite identificar una problemática crítica dentro de la comunidad chilena de juegos de cartas coleccionables, particularmente en el entorno de Magic: The Gathering: la falta de una plataforma unificada que conecte a jugadores, organizadores de torneos y tiendas locales en un mismo ecosistema digital.

Actualmente, los usuarios deben desenvolverse en un entorno fragmentado, donde la información sobre eventos circula por redes sociales informales, la construcción de mazos depende de herramientas aisladas y la gestión de torneos se realiza mediante procesos manuales, planillas o métodos poco estandarizados. Esta situación aumenta la barrera de entrada para nuevos jugadores, dificulta la organización competitiva, limita la visibilidad de las tiendas locales y reduce las posibilidades de crecimiento sostenido de la comunidad.

Frente a este escenario, Deckora se consolidó como una solución web centralizada orientada a unificar y modernizar la experiencia TCG, integrando en un solo entorno tres funciones principales: la gestión y construcción de mazos con asistencia de inteligencia artificial vectorial, la organización completa de torneos con manejo de inscripciones, rondas y resultados, y un directorio público de tiendas con mapa interactivo y visualización de torneos activos.

A diferencia de una propuesta meramente conceptual, el proyecto alcanzó un estado funcional, desplegado y verificado. La plataforma se encuentra operativa sobre una arquitectura real —Vercel para el frontend, Render para el backend y Supabase como base de datos—, con sus módulos de mazos, torneos, rondas, biblioteca y estadísticas implementados de extremo a extremo. Más relevante aún, su corrección no se afirma de manera anecdótica: se respalda con una batería de **1.326 casos de prueba automatizados** que alcanzó un **99,8 % de éxito**, con coberturas de código superiores al **90 % en sentencias** tanto en backend como en frontend, y bajo la regla estricta de que ningún test interactúa con la base de datos de producción. Los dos únicos casos no superados corresponden a límites de la infraestructura de capa gratuita y no a defectos del código, que mantuvo un **0 % de errores HTTP** bajo carga.

El proceso de aseguramiento de calidad no solo confirmó el correcto funcionamiento de la lógica de negocio crítica —validación de mazos, emparejamientos y registro de resultados—, sino que además permitió detectar y documentar hallazgos relevantes antes de exponer el sistema a usuarios reales, como una condición de carrera en inscripciones concurrentes y oportunidades concretas de endurecimiento en seguridad. Esto demuestra que la estrategia de pruebas priorizada por riesgo cumplió su objetivo: entregar confianza sobre lo que funciona y visibilidad clara sobre lo que aún debe mejorarse antes de un uso productivo.

De esta manera, Deckora no solo simplifica la experiencia de los jugadores, sino que también profesionaliza la gestión de torneos locales, entrega mayor visibilidad a las tiendas y fortalece el ecosistema competitivo de Magic: The Gathering en Chile.

22.1 Beneficios esperados

Con la implementación de Deckora —parte de cuyos efectos ya son demostrables sobre la versión desplegada— se proyectan los siguientes impactos positivos para la comunidad:

- **Centralización y eficiencia operativa:** la plataforma reduce la dispersión de información y herramientas, permitiendo que jugadores, organizadores y tiendas accedan a funciones clave desde un mismo lugar. Esto disminuye el tiempo dedicado a buscar eventos, administrar recursos o coordinar torneos.
- **Reducción de fricción para jugadores nuevos:** el asistente de inteligencia artificial basado en embeddings vectoriales permite sugerir cartas compatibles con el estilo de un mazo, sin que el jugador tenga que dominar años de metajuego, reglas tácitas o conocimiento acumulado de cartas.
- **Mejora en el rendimiento competitivo:** gracias a la integración de herramientas de análisis, historial de desempeño y asistencia inteligente, los jugadores pueden tomar decisiones basadas en datos reales para optimizar sus mazos, ajustar estrategias y mejorar progresivamente su nivel de juego.
- **Profesionalización de la gestión de torneos locales:** organizadores y tiendas pueden utilizar una herramienta gratuita para gestionar inscripciones, rondas Swiss, resultados y tablas de posiciones, reemplazando métodos manuales propensos a errores.

- **Reducción de errores en la gestión:** la automatización de enfrentamientos, registros de resultados y clasificación de participantes permite desarrollar torneos más fluidos, transparentes y ordenados.
- **Visibilidad para tiendas locales:** el mapa público de tiendas y el listado de torneos activos permiten que las tiendas sean encontradas por la comunidad sin depender exclusivamente de redes sociales o publicidad pagada.
- **Fortalecimiento del ecosistema local:** al conectar jugadores, tiendas y eventos en una misma plataforma, Deckora fomenta una participación más activa, mejora la comunicación dentro de la comunidad y contribuye al crecimiento regional del juego organizado.
- **Arquitectura escalable a bajo costo:** el stack tecnológico utilizado (Vercel, Render, Supabase y servicios gratuitos o de bajo costo de inteligencia artificial) permite operar la plataforma con un costo cercano a cero durante la etapa de validación, manteniendo posibilidades de escalabilidad futura.

22.2 Próximos pasos

Con la plataforma ya construida, desplegada y validada, los próximos pasos dejan de ser de construcción inicial y se orientan a tres frentes: endurecer el sistema para un uso productivo real, escalar su capacidad y validar su adopción en la comunidad. Cabe destacar que esta hoja de ruta no es especulativa: se desprende directamente de los hallazgos y límites identificados durante el proceso de pruebas documentado en este informe.

22.3 Objetivos a corto plazo: Preparación para producción

- Resolver los hallazgos de seguridad documentados durante las pruebas: incorporar **rate limiting** en los endpoints de autenticación (riesgo **R-09**) mediante express-rate-limit, definir la política de control de acceso para la creación de torneos (hallazgo **R-07**: POST /api/torneos sin restricción de rol) y configurar **CORS** con una whitelist explícita de orígenes permitidos.
- Corregir de forma definitiva la **condición de carrera** detectada en inscripciones concurrentes, asegurando que el sistema responda con el código de conflicto correcto (**409 en lugar de 500**) bajo alta concurrencia, e incorporar una prueba de regresión específica que cubra ese escenario.

- Completar y ejecutar el **checklist previo a producción** definido en el plan de pruebas, dejando trazabilidad de cada punto verificado.

22.4 Objetivos a mediano plazo: Escalabilidad y robustez

- Migrar desde la infraestructura de capa gratuita (Render/Supabase) hacia planes con mayor capacidad, de modo de superar los límites de latencia que provocaron los dos fallos en las pruebas de carga, y volver a ejecutar los escenarios de **k6** para verificar el cumplimiento de los umbrales p95/p99.
- Ampliar la cobertura de pruebas que se difirió en esta etapa: validación **cross-browser** (hoy limitada a Chromium) y pruebas de **accesibilidad (a11y)**, con el fin de asegurar una experiencia inclusiva y consistente entre navegadores.
- Consolidar el pipeline de **CI/CD** como puerta de calidad permanente, manteniendo los gates de auditoría de dependencias, análisis estático y suite completa de pruebas en cada integración hacia las ramas principales.

22.3 Objetivos a largo plazo: Adopción y evolución del producto

- Ejecutar un **piloto** con tiendas y organizadores reales de la comunidad chilena de Magic: The Gathering, recogiendo retroalimentación de uso real para priorizar mejoras y validar la propuesta de valor.
- Evaluar la incorporación de nuevas funcionalidades de alto valor: soporte para formatos y tipos de torneo adicionales, mejoras al asistente de IA y notificaciones en tiempo real.
- Definir un modelo de operación sostenible que preserve el bajo costo de la etapa de validación a medida que la base de usuarios crece.

Con esta hoja de ruta, Deckora se proyecta más allá de un prototipo académico: como una solución viable, escalable y útil para la comunidad chilena de Magic: The Gathering, sustentada en una base técnica ya probada sobre la cual continuar evolucionando.

23. Anexos

- [1.Diagrama de árbol](#)
- [2.Diagrama As-Is perfil de jugador](#)
- [3.Diagrama As-Is perfil de organizador/tienda](#)
- [4.Diagrama de infraestructura](#)
- [5.Diagrama To-Be perfil jugador](#)
- [6.Diagrama To-Be perfil tienda](#)
- [7.Diagrama To-Be perfil organizador](#)
- [8.Tareas en Trello](#)
- [9.Diagrama de casos de uso](#)
- [10.Diagrama de flujo de perfil jugador](#)
- [11.Diagrama de flujo de perfil organizador/tienda](#)
- [12.Landing page: hero](#)
- [13.Landing page: mapa](#)
- [14.Perfil de jugador: dashboard](#)
- [15. Perfil jugador: mis mazos](#)
- [16. Perfil jugador: detalle de mazo](#)
- [17. Perfil jugador: edición de mazo](#)
- [18. Perfil organizador: gestión de torneos](#)
- [19. Perfil organizador: vista de perfil](#)
- [20. Biblioteca: filtro por edición](#)
- [21. Modelo de base de datos](#)
- [22. Diagrama de arquitectura](#)
- [23. Diagrama de componentes](#)
- [24. Estructura de carpetas: backend](#)
- [25. Estructura de carpetas: frontend](#)
- [26. Historial commits: frontend](#)
- [27. historial Pull requests: frontend](#)

- [28. Historial commits: backend](#)
- [29. Historial pull requests: backend](#)
- [30. Render: registro de actividad](#)
- [31. Render: variables de entorno](#)
- [32. Vercel: panel de despliegue](#)
- [33. Supabase: editor de tablas](#)
- [34. Supabase: Extensiones utilizadas](#)
- [35. Deckora desplegado](#)
- [36. Cantidad total de tests backend](#)
- [37. Porcentaje de cobertura de test backend](#)
- [38. Cantidad total de tests frontend](#)
- [39. Porcentaje de cobertura de tests de frontend](#)
- [40. Resultados de tests e2e parte 1](#)
- [41. Resultados de tests e2e parte 2](#)
- [42. Resultado de tests de carga](#)
- [43. Resultado auditoría backend](#)
- [44. Resultado auditoría frontend](#)
- [45. Resultado de análisis de seguridad backend](#)
- [46. Resultado de análisis de seguridad frontend](#)